Ben-Gurion University of the Negev

Faculty of Engineering Sciences

[DEPARTMENT DETAILS REQUIRED]

A DAG-Guided Framework for Proxy-State Effect Estimation in Irregular ICU Time Series

Thesis submitted in partial fulfillment of the requirements for the Master degree

[AUTHOR DETAILS REQUIRED]

[SUPERVISOR DETAILS REQUIRED]

[ADMINISTRATIVE DETAILS REQUIRED]

Approval Page

A DAG-Guided Framework for Proxy-State Effect Estimation in Irregular ICU Time Series

[AUTHOR DETAILS REQUIRED]

[SUPERVISOR DETAILS REQUIRED]

[DEPARTMENT DETAILS REQUIRED]

[APPROVAL TEXT REQUIRED]

[ADMINISTRATIVE DETAILS REQUIRED]

Secondary-Language Abstract

[STAGE 4 DRAFT REQUIRED]

[SUPERVISOR DECISION REQUIRED]

Primary-Language Abstract

[STAGE 4 DRAFT REQUIRED]

[SUPERVISOR DECISION REQUIRED]

Keywords

Primary-Language Keywords

[STAGE 4 DRAFT REQUIRED]

Secondary-Language Keywords

[STAGE 4 DRAFT REQUIRED]

[SUPERVISOR DECISION REQUIRED]

Acknowledgements

[STAGE 4 DRAFT REQUIRED]

[ADMINISTRATIVE DETAILS REQUIRED]

Contents

Abbreviations

[STAGE 4 DRAFT REQUIRED]

Notation and Symbols

[STAGE 4 DRAFT REQUIRED]

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation: Irregular ICU Time-Series Analysis

[STAGE 4 DRAFT REQUIRED]

1.2 Thesis Gap and Objective

[STAGE 4 DRAFT REQUIRED]

1.3 Research Questions and Contributions

[STAGE 4 DRAFT REQUIRED]

[SUPERVISOR DECISION REQUIRED]

1.4 Thesis Organization

[STAGE 4 DRAFT REQUIRED]

[SUPERVISOR DECISION REQUIRED]

Chapter 2

Background and Related Work

2.1 ICU EHR Datasets and Irregular Sampling

[STAGE 4 DRAFT REQUIRED]

2.2 Irregular Time-Series Representation Models

[STAGE 4 DRAFT REQUIRED]

2.3 Proxy Phenotyping and Weak Supervision

[STAGE 4 DRAFT REQUIRED]
[VALIDATION REQUIRED]

2.4 Causal Diagrams, Identification, HTE, Overlap, and Sensitivity

[STAGE 4 DRAFT REQUIRED]
[CITATION REQUIRED]

Chapter 3

Problem Definition and Study Design

3.1 Units, Time Horizons, and Data Objects

This thesis studies retrospective intensive-care records represented as irregular clinical time series. The operational study unit is a time-series record or ICU stay, depending on the source dataset and preprocessing path. PhysioNet 2012 source files use record identifiers, whereas MIMIC-III source tables use ICU-stay and admission identifiers. The implemented pipelines normalize these source-specific identifiers to the canonical join key `ts_id`. This identifier should be read as the analysis-record identifier used by the software; it is not assumed to be a globally unique person identifier across repeated admissions or stays.

For a study unit i, the observed longitudinal data are represented as a finite collection of measurement events

$$\mathcal{O}_i = \{(i, t_{ij}, v_{ij}, x_{ij}) : j = 1, \ldots, n_i\},$$

where i is the normalized `ts_id`, t_{ij} is elapsed time in minutes from record start or ICU admission, $v_{ij} \in \mathcal{V}$ is the observed variable name, and x_{ij} is the numeric value recorded for that variable. In the causal repository this long-format object is stored in `ts` with the required columns `ts_id`, `minute`, `variable`, and `value`. Static or baseline quantities such as age, sex or gender coding, height, weight, and ICU-type indicators are represented within the same event schema as variables observed at or near the beginning of the record when the preprocessing implementation provides them.

The event representation is intentionally irregular. Observations occur at nonuniform times, different variables are measured with different frequencies, and most variables are absent at most possible times. A missing measurement is therefore not interpreted as a normal value. It may reflect a clinical decision not to measure, an unrecorded observation, a source-table limitation, or a preprocessing exclusion. The thesis treats informative measurement and missingness processes as important threats to interpretation, not as causal findings established by the repository.

The primary patient- or stay-level outcome used by the downstream causal repository is `in_hospital_mortality`. In the causal processed artifact, this outcome is stored in `oc`, the outcome table keyed by `ts_id`. The third object in that processed artifact, `ts_ids`, is the sorted collection of normalized identifiers represented in `ts`. Later predictive processing uses a

distinct split-aware artifact and must not be conflated with this causal contract.

3.2 Prediction Task

The prediction task is a multi-label proxy-state prediction problem over irregular ICU time series. For each study unit i and proxy state k, let $L_{ik}^{rule} \in \{0, 1\}$ denote a rule-derived proxy label. These labels are clinically inspired analytical constructs generated from observed data and deterministic source-code rules. They are weak clinical labels or proxy phenotypes, not chart-adjudicated diagnoses, manually verified clinical states, or validated phenotypes unless separate validation evidence is supplied in a later chapter.

The supervised predictive workflow estimates a vector of proxy-state labels from the observed time-series representation. Its target matrix is loaded from a latent-label CSV keyed by `ts_id`; the supervised proxy-label targets are not taken from the mortality column in `oc`. For model m, the exported prediction object may contain probabilities $\hat{p}_{ik}^{(m)}$ and corresponding binary predicted labels $\hat{L}_{ik}^{(m)}$. These outputs are model estimates of rule-derived proxy labels rather than direct measurements of patient physiology.

At the level of study design, five analytical tasks are distinguished. First, rule-based proxy-state construction derives binary proxy labels from observed clinical measurements. Second, multi-label proxy-state prediction estimates those rule-derived labels from irregular time-series inputs. Third, proxy-label aggregation may combine binary outputs from several voter sources into a single proxy-state table. Fourth, mortality prediction or association analysis evaluates whether proxy-state representations contain prognostic information about `in_hospital_mortality`. Fifth, observational causal-effect estimation treats selected proxy-state indicators as modeled exposures and estimates adjusted contrasts under explicit assumptions. Detailed proxy rules, model architectures, estimator algorithms, and numerical results are intentionally deferred to later chapters.

3.3 Causal Question, Exposures, Outcome, Estimands, Assumptions

The downstream causal-analysis setting is retrospective and observational. No intervention is assigned by this study. The repository uses software parameters named `treatment` for selected binary `LAT_*` columns, but many such columns represent illness-state or organ-dysfunction proxy states rather than well-defined administered therapies. In the thesis narrative these variables are therefore described as proxy-state exposures or modeled treatment variables unless a later section explicitly defines a manipulable intervention.

Let $A_i \in \{0, 1\}$ denote a selected proxy-state exposure, Y_i denote the binary outcome `in_hospital_mortality`, W_i denote observed adjustment variables selected from a project-specified DAG and available columns, and X_i denote effect-modifier features used to describe heterogeneity. The dataset-specific DAGs were developed through a project design process that

included LLM-assisted clinical and causal elicitation and were then encoded in source code; the implemented graph scripts, not the prompt outputs alone, define the DAG artifacts used by the pipeline. Potential-outcome notation such as $Y_i(a)$ is useful as conceptual language, but causal interpretation requires more than the presence of a graph or an estimator. It depends on assumptions about the causal graph, consistency, exchangeability conditional on observed covariates, positivity or overlap, outcome and exposure measurement, and unmeasured confounding [1, 2, 3].

Accordingly, later estimates should be interpreted as adjusted effect estimates under project assumptions, not as unconditional clinical effects. The existence of a DAG, matching script, or CATE estimator does not prove identification. Proxy states may be useful for structured analysis while still carrying label noise, temporal ambiguity, and intervention-definition limitations.

[SUPERVISOR DECISION REQUIRED: final estimand wording for proxy-state exposures and aggregated CATE summaries]

The approved main research question is how irregular ICU time-series data can be converted into clinically interpretable proxy-state representations and used, under explicit causal assumptions, to support DAG-guided adjusted effect estimation for in-hospital mortality. Within the scope of this chapter and the next, the relevant subquestions are whether rule-derived clinical proxy states can be predicted from irregular ICU time series, which data contracts are needed to connect PhysioNet 2012, MIMIC-III, STraTS, and the causal-analysis pipeline, whether proxy-state representations contain mortality-related information, how adjusted estimates vary across proxy-state exposures or patient characteristics, and how robust those estimates are to modeling and confounding assumptions. These are study questions and design commitments; their answers are reserved for the results and discussion chapters.

Chapter 4

Data and Preprocessing

4.1 PhysioNet 2012 Pipeline

The PhysioNet/Computing in Cardiology Challenge 2012 dataset provides ICU time-series records and associated in-hospital mortality outcomes for a mortality-prediction challenge setting [4]. In this thesis it functions as one of the two ICU time-series sources used to exercise the proxy-state prediction and downstream causal-analysis pipeline. The local repository does not track the raw challenge files or a verified processed-data pickle, so this section describes the implemented preprocessing contract rather than asserting final cohort counts.

[RESULT REQUIRED: final number of included PhysioNet records after preprocessing]

The causal-repository PhysioNet preprocessing implementation traverses the raw `set-a`, `set-b`, and `set-c` folders and reads one patient-record file at a time. It removes rows with missing parameter names, skips records with at most five retained measurement rows, and removes observations with negative values, which the source code treats as missingness indicators. It converts source times from `HH:MM` strings into elapsed minutes, renames `RecordID` to `ts_id`, `Parameter` to `variable`, and `Value` to `value`, and reads outcome files by mapping `In-hospital_death` to `in_hospital_mortality`. After concatenating the source sets, it drops duplicate events and converts the categorical `ICUType` measurement into one-hot-style variables `ICUType_1` through `ICUType_4`.

The resulting causal processed artifact is serialized as three objects: `ts`, `oc`, and `ts_ids`. The `ts` object is a long event table with `ts_id`, `minute`, `variable`, and `value`. The `oc` object contains `ts_id`, `length_of_stay`, `in_hospital_mortality`, and a source subset indicator. The `ts_ids` object is derived from identifiers observed in `ts`. This artifact is the expected input to rule-based tagging, mortality-from-proxy analysis, matching, and CATE estimation in the causal repository.

4.2 MIMIC-III Pipeline

MIMIC-III is a critical-care electronic health-record database used here as the second ICU data source [5]. The implemented workflow follows a clinical time-series benchmark style in that it

transforms raw ICU events into stay-level time-series examples suitable for prediction, while adapting the output contract for this thesis pipeline [6]. The repository does not establish a tracked raw-data snapshot, extraction date, or final processed artifact hash.

[RESULT REQUIRED: final number of included MIMIC-III ICU stays after preprocessing]

The active causal-repository MIMIC preprocessing implementation reads broad categories of MIMIC-III source tables: ICU-stay timing from `ICUSTAYS`, demographics from `PATIENTS`, bedside measurements from `CHARTEVENTS`, laboratory measurements from `LABEVENTS`, fluid outputs from `OUTPUTEVENTS`, administered inputs from `INPUTEVENTS_CV` and `INPUTEVENTS_MV`, and mortality/timing information from `ADMISSIONS`. Large event tables are processed in chunks and written to temporary shards before feature rows are collected. The implementation filters to adult ICU stays, removes chart events marked with an error flag, requires valid chart times and numeric or textual values, applies item-ID mappings and range filters for selected variables, converts selected units, and assigns events without direct ICU-stay identifiers to an ICU interval when possible.

For the causal contract, event times are converted to elapsed minutes relative to ICU admission, events are restricted to the early ICU window used by the implementation, age and gender are added as static rows at minute zero, and duplicate (`ts_id`, `minute`, `variable`) events are collapsed by averaging. The helper contract canonicalizes stay identifiers, normalizing numeric-looking identifiers such as `12345.0` to `12345`. It defines the canonical `ts` columns as `ts_id`, `minute`, `variable`, and `value`, defines the canonical `oc` columns as `ts_id`, `length_of_stay`, `in_hospital_mortality`, and `subset`, and intentionally excludes internal MIMIC identifiers such as `HADM_ID` and `SUBJECT_ID` from the exported outcome table.

Inspection found a source-level issue that should be resolved before relying on local MIMIC preprocessing execution: the active script reads `ADMISSIONS.csv` with `DEATHTIME` for early-death filtering, while the canonical outcome helper requires `HOSPITAL_EXPIRE_FLAG` to construct `in_hospital_mortality`. This does not change the documented target contract, but it is a reproducibility risk recorded in the Stage 4.1 evidence report and deferred-fix register.

4.3 STraTS Split-Aware Artifacts Versus Causal Artifacts

The predictive STraTS repository uses a different processed-data contract from the causal repository. Its loader expects a five-object pickle consisting of `events`, `oc`, `train_ids`, `val_ids`, and `test_ids`, where the final three objects explicitly encode train, validation, and test identifiers. The loader accepts stay identifiers named `ts_id`, `icustay_id`, or `ICUSTAY_ID`, canonicalizes them to `ts_id`, and validates the split arrays after the same normalization. This split-aware contract is therefore not interchangeable with the causal repository's unsplit `[ts, oc, ts_ids]` artifact.

In supervised STraTS runs, the latent-label CSV is joined to the selected split identifiers after ID normalization. The loader filters labels to the supervised IDs, raises an error when

labels are missing for any supervised `ts_id`, raises an error for duplicate normalized labels, and uses all non-`ts_id` columns in the latent CSV as proxy-label targets. The mortality column in `oc` is not used as the supervised proxy-label target. Prediction export writes `ts_id`, one probability column per proxy state, and one thresholded binary column per proxy state; the wrapper scripts inspected for this stage export the `all` split for their prediction CSVs, although final archive-copy provenance remains incomplete.

The predictive loader also implements split-aware leakage controls. It keeps only variables observed in the training split, derives normalization statistics from the training split, computes static-feature normalization from training rows, and validates label alignment before training. For MIMIC-III supervised prediction it restricts events to the first 24 hours. The pretraining path removes test data, uses training-derived variable statistics, uses a 48-hour maximum window for PhysioNet 2012, and permits MIMIC-III observations up to five days while sampling 24-hour input windows. These controls reduce some sources of leakage, but they do not guarantee absence of leakage, nor do they resolve missing raw-data and split-provenance limitations.

The parent router can bridge contracts by reading a causal `[ts, oc, ts_ids]` artifact and constructing a STraTS-style `[ts, oc, train_ids, val_ids, test_ids]` payload through a seeded identifier shuffle. That bridge is an orchestration convenience and does not prove how every archived final STraTS artifact was generated.

Table 4.1: Processed artifact contracts used by the causal and STraTS repositories.

Property	Causal repository	STraTS repository
Processed artifact	`[ts, oc, ts_ids]`	`[events, oc, train_ids, val_ids, test_ids]`
Split representation	Unsplit ID collection	Explicit train/validation/test IDs
Main use	Tagging and causal analysis	Predictive training and export
Identifier convention	Canonical `ts_id`	Loader-normalized `ts_id`
Proxy-label source	Separate latent CSV downstream	Separate latent CSV joined to split IDs

[FIGURE REQUIRED: audited dataflow diagram linking raw data, causal artifacts, STraTS artifacts, proxy labels, and causal analysis]

Several provenance limitations remain. Raw PhysioNet and MIMIC-III data are not tracked in Git. The processed pickles used by final runs are external or absent from the audited repository. Some run records contain machine-specific absolute paths, and final cohort counts require verified processed artifacts or manifests. Clean-checkout reproduction therefore requires authorized data access, artifact hashes, and a documented split-generation record.

[VALIDATION REQUIRED: checksums for processed PhysioNet and MIMIC-III dataset artifacts]

[VALIDATION REQUIRED: provenance of train/validation/test split generation for final STraTS artifacts]

Chapter 5

Clinical Proxy-State Construction

This chapter describes how the repository transforms heterogeneous, irregular ICU measurements into a smaller set of interpretable binary constructs. The constructs provide supervised targets for the proxy-state prediction task introduced in Chapter 3 and structured proxy-state exposures for later observational analyses. Their interpretability comes from explicit source-code rules and from stable `LAT_*` column names, not from chart-adjudicated clinical reference labels. They should therefore be read as proxy states: useful analytical representations whose value depends on construct validity, source-variable availability, measurement quality, missingness, and later clinical review.

The chapter is limited to methods and construct definition. It documents the active rule implementations for PhysioNet 2012 and MIMIC-III, describes available validation artifacts without reporting their numerical contents, and explains how rule-derived labels differ from model-predicted and majority-vote proxy states. Predictive-model architecture and training mechanics are deferred to Chapter 6; final numerical results are deferred to later results chapters.

5.1 Rationale and Terminology

Electronic health-record phenotyping often uses structured measurements, codes, and rules to construct clinically meaningful variables when direct expert labels are unavailable [7]. This thesis uses the same conservative framing. A *proxy state* is a binary construct intended to summarize evidence for a clinically meaningful condition or physiologic state from the available data. A *proxy phenotype* is the same family of construct, with emphasis on phenotype-like grouping rather than direct measurement. A *clinically inspired proxy label* is a weak label based on clinical measurements, thresholds, or rule clauses rather than a chart-adjudicated endpoint. A *rule-based proxy state* is assigned by deterministic source-code rules. A *predicted proxy state* is produced by a supervised time-series model trained to estimate rule-derived labels. A *majority-vote proxy state* is the deterministic aggregation of binary voter files. A *binary proxy-state tag* is the realized 0/1 value for one `ts_id` and one proxy-state column.

The repository historically uses the names `latent`, `latent variable`, `latent tag`, and `LAT_*`. Those names are stable implementation vocabulary. The thesis retains the `LAT_*` col-

umn names when identifying artifacts, configuration fields, prediction targets, and downstream exposure variables, but uses "proxy state" in explanatory prose. This avoids implying that the implementation has discovered or validated a hidden biologic state.

The proxy-state layer serves four methodological roles. First, it compresses multivariate measurements into interpretable, patient- or stay-level constructs. Second, it supplies weak or rule-derived supervised targets for multi-label prediction. Third, it establishes a consistent variable interface for downstream mortality, matching, and CATE scripts. Fourth, it preserves a transparent mapping from each `LAT_*` column back to measurement families and rule clauses.

The limitations are equally central. No chart-adjudicated reference labels or clinician-reviewed gold-standard labels were found in the inspected archive. Thresholds approximate clinical ideas but are not equivalent to complete formal clinical definitions. Missing measurements can prevent rule clauses from firing; a negative tag therefore does not prove absence of the condition, and a positive tag does not establish a clinical diagnosis. Source-variable availability differs between PhysioNet and MIMIC-III, so similarly named cross-dataset states are not automatically identical constructs. Weak-supervision literature supports the broad idea that noisy labeling sources and aggregation can be useful, but this project does not implement the Snorkel generative label model [8].

5.1.1 LLM-Assisted Ontology and Rule Elicitation

The proxy-state ontology and dataset-specific DAGs also have prompt-provenance artifacts under `thesis-writing/prompts-and-documents/`. Those artifacts document an LLM-assisted elicitation process used to propose candidate latent/proxy clinical states, rule families, missingness and measurement-process considerations, and DAG structures for PhysioNet 2012 and MIMIC-III. The final dataset prompts instantiate a generic prompt template with only the dataset name and official dataset-description URL changed. The exported prompt runs are documented, based on user-supplied project metadata, as ChatGPT 5.4 with extended reasoning; the PDF exports themselves record browser-export metadata but do not independently encode the model-version field.

This prompt layer is treated as design provenance rather than clinical validation or source-code authority. The LLM did not learn labels or graphs from patient-level records in this repository, did not execute the preprocessing, prediction, majority-vote, or causal-estimation pipeline, and did not produce chart-adjudicated reference labels. The implemented rule-derived proxy states are defined by the active tagger source files, and the implemented DAG artifacts are defined by the active graph scripts. The prompt outputs are therefore useful for explaining how candidate constructs were elicited and refined, but any final thesis claim about implemented behavior must trace to source code, configuration, and run artifacts.

[VALIDATION REQUIRED: record the final prompt execution settings, research or browsing mode, follow-up prompts, and output-export procedure for the ChatGPT 5.4 extended-reasoning runs]

[SUPERVISOR DECISION REQUIRED: document the human and clinical review applied to

Table 5.1: Prompt and document artifacts used as design provenance for proxy-state and DAG construction.

Artifact	Dataset	Role	Version status	Thesis relation
Generic prompt template	Generic	Template for staged ontology, rule, missingness, and DAG elicitation.	Final template	Prompt-engineering provenance only.
PhysioNet prompt and run export	PhysioNet	Dataset-specific prompt and exported ChatGPT run.	Final run	Candidate design proposal; not implementation authority.
MIMIC prompt and run export	MIMIC-III	Dataset-specific prompt and exported ChatGPT run.	Final run	Candidate design proposal; not implementation authority.
Earlier prompt-result exports	Both	Earlier prompt outputs.	Superseded	Historical design trace only.
Manager-summary documents	Project	Management summaries of proxy/DAG and clinical CATE review concepts.	Review documents	Human/project review context; not source execution.

the LLM-generated proxy-state and DAG design proposals]

[SUPERVISOR DECISION REQUIRED: approve proxy state as the primary thesis term and review construct-level clinical wording]

5.2 PhysioNet Proxy-State Rules

The active PhysioNet implementation is `tagging_latent_variables_physionet.py` in the causal repository's `src/` directory. It loads the processed PhysioNet pickle, pivots the long table by `ts_id` and minute, computes per-record summary columns using minimum, maximum, mean, first observed value, and last observed value, and then applies one decision function per configured `LAT_*` column. The active output order is:

`LAT_CHRONIC_BASELINE_RISK`, `LAT_GLOBAL_SEVERITY`, `LAT_SHOCK`, `LAT_RESPIRATORY_FAILURE`, `LAT_RENAL_DYSFUNCTION`, `LAT_HEPATIC_DYSFUNCTION`, `LAT_COAG_HEME_DYSFUNCTION`, `LAT_INFLAMMATION_SEPSIS_BURDEN`, `LAT_NEUROLOGIC_DYSFUNCTION`, `LAT_CARDIAC_INJURY_STRAIN`, `LAT_METABOLIC_DERANGEMENT`.

The archived `final-results/trees/physionet-tags/physionet-latent-tags.csv` header matches this active column set and contains binary values with no duplicate `ts_id` values in the audit. That artifact-level agreement supports the schema, but it does not by itself prove the producing command, input-artifact hash, source commit, or default-versus-optimized threshold mode.

Missingness is handled conservatively in the active decision helpers. A comparison such as

Table 5.2: Proxy-state source types used in the thesis pipeline.

Source type	Generation mechanism	Interpretation boundary	Primary evidence
Rule-derived proxy state	Deterministic clinical rules applied to summary features or helper flags.	Transparent weak label; not a verified diagnosis or formal score unless the full score is implemented.	Active tagger source and rule artifacts.
Predicted proxy state	Time-series model probability thresholded at 0.5 for each target column.	Model estimate of a rule-derived proxy label; not a direct clinical measurement.	STraTS export source and prediction CSV schema.
Majority-vote proxy state	Deterministic vote across binary voter CSVs aligned on shared `ts_id`.	Aggregated binary proxy; not clinical consensus and not a probabilistic label model.	Majority-vote source and causal run summaries.

$x < c$ or $x \geq c$ contributes positive evidence only when a non-missing value exists. Missing values therefore do not count as abnormal. The active summarizer does not implement explicit time windows beyond whatever time span is already present in the processed PhysioNet record. It also does not compute a fresh urine sum; urine-related clauses fire only if a compatible precomputed summary column such as `Urine_24h_min`, `Urine_24h_sum`, or `Urine_sum` is present. For oxygenation, the implementation uses `PF_min` when available and otherwise approximates the PaO2/FiO2 ratio from PaO2 and FiO2 summary columns.

By default the script uses the clinically inspired thresholds embedded in the active source. The command-line option `--optimized` switches to externally supplied thresholds loaded from `--thresholds-path`, with unspecified optimized keys falling back to defaults. The existence of this option is not evidence that optimized thresholds generated the archived CSV.

[VALIDATION REQUIRED: identify whether the archived PhysioNet proxy-state CSV used default or externally optimized thresholds and record the threshold-file provenance]

Table 5.3: PhysioNet rule-derived proxy-state definitions from the active tagger.

Proxy-state column	Construct represented	Input measurements and aggregation	Rule structure or threshold family	Grounding and validation status
`LAT_CHRONIC_BASELINE_RISK`	Baseline vulnerability or limited reserve.	Age first value; BMI from height and weight first values; albumin minimum; ICU type first value.	Score at least 2 across age ≥ 75, BMI < 18.5 or ≥ 40, albumin < 3.0, or ICU type in $\{1, 3\}$.	Phenotyping context [7]; exact clauses are project-specific.

Proxy-state column	Construct represented	Input measurements and aggregation	Rule structure or threshold family	Grounding and validation status
LAT_GLOBAL_SEVERITY	Multi-domain acute physiologic severity.	Hemodynamic, respiratory, neurologic, renal, hepatic/coagulation, metabolic, cardiac summaries: minima, maxima, and helper flags.	At least 3 abnormal domains, or at least 2 domains plus a critical marker: lactate ≥ 4.0, pH < 7.20, mechanical ventilation, GCS ≤ 8, or MAP < 60. Domain thresholds include MAP < 70, PF < 300, GCS < 15, creatinine ≥ 2.0, bilirubin ≥ 2.0, platelets < 100, lactate > 2.0, and HR ≥ 130.	Broad organ-dysfunction grounding from Sepsis-3 and SOFA [9, 10]; not complete SOFA.
LAT_SHOCK	Circulatory shock or hemodynamic instability.	MAP, noninvasive MAP, systolic pressure, lactate, heart rate, urine helper, pH, bicarbonate.	At least 2 abnormal clauses among MAP < 70, SBP ≤ 90, lactate > 2.0, HR ≥ 110, urine < 500, pH < 7.30, or bicarbonate < 18; also positive for low MAP with lactate ≥ 4.0.	Shock/lactate concepts grounded by Sepsis-3 [9]; no vasopressor requirement in PhysioNet rule.
LAT_RESPIRATORY_FAILURE	Hypoxemia, ventilatory failure, or ventilatory support.	Mechanical ventilation flag; PF ratio; SaO2; PaO2; respiratory rate; PaCO2; pH.	At least 2 abnormal clauses or mechanical ventilation with PF < 300. Clauses include SaO2 < 92, PaO2 < 60, respiratory rate ≥ 22 or < 8, PaCO2 ≥ 50, or PaCO2 ≥ 45 with pH < 7.30.	Respiratory phenotyping and ARDS/oxygenation grounding [11, 12]; not adjudicated ARDS.
LAT_RENAL_DYSFUNCTION	Renal dysfunction.	Creatinine first and maximum; BUN maximum; urine helper; potassium maximum; bicarbonate minimum.	At least 2 abnormal clauses among creatinine ≥ 2.0, creatinine rise ≥ 0.3, BUN ≥ 40, urine < 500, potassium ≥ 5.5, or bicarbonate < 18; creatinine ≥ 3.5 is sufficient.	Renal concepts grounded by KDIGO [13]; not full KDIGO staging.
LAT_HEPATIC_DYSFUNCTION	Hepatic injury or reserve.	Bilirubin, AST, ALT, ALP, albumin, platelets.	Weighted score at least 2: bilirubin ≥ 2.0 counts twice; AST or ALT ≥ 200, ALP ≥ 250, albumin < 2.5, or platelets < 100 add one.	Bilirubin/organ-dysfunction grounding from SOFA [10]; transaminase and albumin clauses are project-specific.
LAT_COAG_HEME_DYSFUNCTION	Hematologic/coagulation stress.	Platelets, hematocrit, WBC, platelet first-to-minimum change.	Score at least 2. Platelets < 100 counts twice; platelets < 150, HCT < 25 or > 55, WBC < 4 or > 20, or platelet drop ≥ 50 add one.	Coagulation grounding from ISTH DIC and SOFA [14, 10]; not complete DIC score.

Proxy-state column	Construct represented	Input measurements and aggregation	Rule structure or threshold family	Grounding and validation status
LAT_INFLAMMATION_SEPSIS_BURDEN	Systemic inflammation or sepsis-like burden.	Temperature, WBC, HR, respiratory rate, PaCO2, lactate, platelets.	Score at least 3 across temperature > 38.3 or < 36, WBC > 12 or < 4, HR > 90, respiratory rate > 20 or PaCO2 < 32, lactate > 2.0, platelets < 150; also positive when temperature or WBC abnormality anchors lactate stress with score at least 2.	Sepsis context [9]; rule is not formal Sepsis-3 diagnosis.
LAT_NEUROLOGIC_DYSFUNCTION	Depressed consciousness or neurologic/metabolic stress.	GCS; sodium; glucose; PaCO2; SaO2; pH.	Score at least 2. GCS ≤ 12 counts twice; GCS < 15, sodium < 130 or > 150, glucose < 70 or > 300, PaCO2 ≥ 50, SaO2 < 90, or pH < 7.25 contribute.	SOFA neurologic context [10]; sedation ambiguity remains unreviewed.
LAT_CARDIAC_INJURY_STRAIN	Myocardial injury, ischemia, or severe strain.	Troponin I/T; ICU type; HR; MAP; systolic pressure; lactate.	Score at least 2. Troponin I > 0.1 or troponin T > 0.01 counts twice; cardiac ICU type, HR ≥ 130 or < 50, MAP < 70 or systolic ≤ 90, and lactate > 2.0 add one.	Project-specific cardiac proxy.
LAT_METABOLIC_DERANGEMENT	Acid-base, lactate, electrolyte, or glucose derangement.	pH, bicarbonate, lactate, sodium, potassium, magnesium, glucose, PaCO2.	Score at least 2 across pH < 7.30 or > 7.50, bicarbonate < 18 or > 32, lactate > 2.0, sodium < 130 or > 150, potassium < 3.0 or > 5.5, magnesium < 0.6 or > 1.2, glucose < 70 or > 250, PaCO2 < 32 or > 50; critical pH < 7.20, lactate ≥ 4.0, or potassium ≥ 6.0 is sufficient.	Project-specific metabolic proxy.

The remaining PhysioNet citation gaps are explicit evidence gates rather than hidden assumptions.

[CITATION REQUIRED: clinical grounding for PhysioNet chronic baseline-risk BMI, albumin, and ICU-type clauses]

[CITATION REQUIRED: clinical grounding for hepatic transaminase, alkaline phosphatase, and albumin clauses in proxy-state rules]

[CITATION REQUIRED: clinical grounding for cardiac injury or strain proxy thresholds]

[CITATION REQUIRED: clinical grounding for metabolic, electrolyte, and acid-base proxy thresholds]

Decision-tree pickle files and rendered tree images exist under the inspected archive and repository tree directories. However, the figure-generation path depends on saved callable objects and plotting code, and the active PhysioNet script's CLI tree-save path requires provenance review before the images can be treated as thesis figures. The rules in Table 5.3 therefore come from the active source and CSV schema rather than from unpickled tree artifacts.

[VALIDATION REQUIRED: verify the provenance and active-rule correspondence of the planned proxy-state decision-tree figures]

5.3 MIMIC Proxy-State Rules and Validation Artifacts

The active MIMIC implementation is `tagging_latent_variables_mimiciii.py` in the causal repository's `src/` directory. Its active column set differs from PhysioNet:

`LAT_CHRONIC_BURDEN`, `LAT_INFLAMMATION_SEPSIS`, `LAT_GLOBAL_SEVERITY`, `LAT_SHOCK`, `LAT_RESPIRATORY_FAILURE`, `LAT_RENAL_DYSFUNCTION`, `LAT_HEPATIC_COAG_DYSFUNCTION`, `LAT_NEUROLOGIC_DYSFUNCTION`, `LAT_METABOLIC_DERANGEMENT`, `LAT_CARDIAC_STRAIN`.

The MIMIC archive header matches this set. Compared with PhysioNet, MIMIC uses a chronic-burden column rather than chronic-baseline-risk, combines hepatic and coagulation evidence into one construct, shortens the inflammation/sepsis and cardiac-strain names, and changes several rule inputs. These naming differences are therefore construct and source-feature differences, not just cosmetic renaming.

The script supports three mutually exclusive input modes. In summary-CSV mode, a pre-computed one-row-per-stay table is loaded; accepted identifier columns include `icustay_id`, `patient_id`, or `stay_id`. In canonical-pickle mode, the script reads a PhysioNet-compatible MIMIC payload [`ts`, `oc`, `ts_ids`], canonicalizes `ts_id`, reconstructs summary features from long events, derives GCS minima from `GCS_eye`, `GCS_motor`, and `GCS_verbal`, aggregates first-24-hour urine output and rolling 6-hour ml/kg/hour urine rates when weight is available, and merges optional outcome/context fields from `oc`. Aliases include `MBP` to MAP, `PCO2` to PaCO2, `PO2` to PaO2, `O2 Saturation` to SpO2, `Creatinine Blood` to creatinine, and `Bilirubin (Total)` to bilirubin. In raw-concept mode, the admissions table is required, and optional diagnoses, vitals, labs, urine, vasopressor, ventilation, infection, and troponin-map CSVs are aggregated into the summary table.

Binary helper flags are treated as evidence only when they are positive or truthy. Numeric comparisons require non-missing values. Raw-concept mode fills absent binary helper columns with zero, while pickle mode inserts missing helper columns as missing; in both cases missing numeric evidence does not make a rule positive. Consequently, input mode and concept extraction completeness affect which clauses can fire.

[VALIDATION REQUIRED: identify the MIMIC tagger input mode, source artifact hash, producing command, and source commit for the archived tag CSV]

Table 5.4: MIMIC rule-derived proxy-state definitions from the active tagger.

Proxy-state column	Construct represented	Input measurements and aggregation	Rule structure or threshold family	Grounding and validation status
LAT_CHRONIC_BURDEN	Baseline chronic burden or vulnerability.	Age; deidentified-old flag; comorbidity count; Elixhauser score; chronic ICD/organ disease helpers; malignancy or immunosuppression; admission type.	Score at least 2 across age $\geq$ 75 or old-age flag, comorbidity count $\geq$ 2 or Elixhauser $\geq$ 5, chronic disease helper, malignancy/immunosuppression helper, or emergency/urgent admission.	Electronic phenotyping context [7]; exact mix is project-specific and input-mode dependent.
LAT_INFLAMMATION_SEPSIS	Inflammation, suspected infection, or sepsis-like burden.	Temperature, WBC, lactate, culture/suspected-infection flags, antibiotic/suspected-infection flags.	Score at least 2 across temperature $\geq$ 38.3 or $<$ 36, WBC $>$ 12 or $<$ 4, lactate $>$ 2.0, culture evidence, or antibiotic evidence; requires infection/inflammation anchoring by culture, antibiotic, temperature, or WBC evidence.	Sepsis context [9]; not formal Sepsis-3 diagnosis.
LAT_GLOBAL_SEVERITY	Multi-domain acute physiologic severity.	Circulatory, respiratory, neurologic, renal, metabolic, inflammatory, hepatic/coagulation domains from summaries and helper flags.	At least 3 domains. Clauses include MAP $<$ 70, SBP $\leq$ 100, vasopressors, SpO2 $<$ 92, PF $\leq$ 300, mechanical ventilation, GCS $<$ 15, RASS ≤ -3 or $\geq$ 2, creatinine $\geq$ 2.0, urine $<$ 500, lactate $>$ 2.0, pH $<$ 7.30, bicarbonate $<$ 18, inflammatory thresholds, bilirubin $\geq$ 2.0, platelets $<$ 100, or INR $\geq$ 1.5.	Broad Sepsis-3/SOFA grounding [9, 10]; not complete SOFA.
LAT_SHOCK	Shock or hemodynamic instability.	MAP, SBP, sustained hypotension helper, vasopressors, lactate, urine output, pH, base excess.	Score at least 2, or vasopressor use plus tissue hypoperfusion, acidosis, or hypotension. Thresholds include MAP $<$ 65, SBP $\leq$ 90, lactate $>$ 2.0, urine $<$ 500 in 24 hours or $<$ 0.5 ml/kg/hour over 6 hours, pH $<$ 7.30, base excess ≤ -5.	Shock/lactate grounding [9]; proxy rule remains source-specific.
LAT_RESPIRATORY_FAILURE	Respiratory failure or respiratory support.	SpO2, PF ratio, FiO2, mechanical/noninvasive ventilation, respiratory rate, PaCO2, pH.	Score at least 2, or respiratory support plus hypoxemia, oxygenation failure, or ventilatory failure. Thresholds include SpO2 $<$ 92, PF $\leq$ 300, FiO2 $\geq$ 0.5, RR $\geq$ 30 or $\leq$ 8, and PaCO2 $\geq$ 50 with pH $<$ 7.30.	Respiratory phenotyping/ARDS context [11, 12]; not adjudicated ARDS.
LAT_RENAL_DYSFUNCTION	Renal dysfunction.	Creatinine maximum and delta, BUN, urine output, potassium, bicarbonate, dialysis/CRRT helper.	Score at least 2, or dialysis/CRRT present. Clauses include creatinine $\geq$ 2.0 or delta $\geq$ 0.3, BUN $\geq$ 40, low urine output, potassium $\geq$ 5.5, or bicarbonate $<$ 18.	KDIGO grounding for creatinine/urine concepts [13]; not full KDIGO staging.
LAT_HEPATIC_COAG_DYSFUNCTION	Combined hepatic and coagulation dysfunction.	Bilirubin, AST, ALT, platelets, INR, PT/PTT helpers, albumin.	Score at least 2 across bilirubin $\geq$ 2.0, AST or ALT $\geq$ 120, platelets $<$ 100, INR $\geq$ 1.5 or PT/PTT prolonged, or albumin $<$ 2.5.	SOFA and ISTH DIC context [10, 14]; combined construct needs clinical review.

Proxy-state column	Construct represented	Input measurements and aggregation	Rule structure or threshold family	Grounding and validation status
LAT_NEUROLOGIC_DYSFUNCTION	Neurologic dysfunction with sedation caveat.	GCS, GCS component abnormality, RASS, pupil/focal-neurologic helper, sedation/intubation helper.	Score at least 2. GCS $\leq$ 8 counts twice; GCS $\leq$ 13, abnormal GCS component, RASS ≤ -3 or ≥ 2, and pupil/focal abnormality contribute. A single sedation/intubation-confounded point is not sufficient.	SOFA neurologic context [10]; sedation ambiguity remains.
LAT_METABOLIC_DERANGEMENT	Acid-base, lactate, electrolyte, or glucose derangement.	pH, bicarbonate, base excess, lactate, potassium, sodium, glucose.	Score at least 2 across pH < 7.30 or > 7.55, bicarbonate < 18 or > 35, base excess ≤ -5, lactate > 2.0, potassium < 3.0 or ≥ 5.5, sodium < 130 or > 150, or glucose < 70 or > 250.	Project-specific metabolic proxy.
LAT_CARDIAC_STRAIN	Cardiac strain or injury.	Troponin flags and values, CK-MB, arrhythmia helper, HR, hypotension, cardiac care unit or surgery context.	Score at least 2 and requires biomarker or rhythm evidence. Biomarker clauses include troponin-positive flags, troponin T ≥ 0.1, troponin I ≥ 0.4, or CK-MB evidence; rhythm/hemodynamic clauses include arrhythmia, HR > 150 or < 40, HR stress with hypotension, or cardiac-care context.	Project-specific cardiac proxy.

The remaining MIMIC citation gaps are explicit evidence gates rather than hidden assumptions.

[CITATION REQUIRED: clinical grounding for MIMIC chronic burden rule clauses]

[CITATION REQUIRED: clinical grounding for MIMIC metabolic, electrolyte, and acid-base proxy thresholds]

[CITATION REQUIRED: clinical grounding for MIMIC cardiac strain proxy thresholds]

The MIMIC output directory contains a binary tag CSV, a merged feature table, prevalence and co-occurrence summaries, mortality-association summaries, a JSON validation summary, and a decision-tree pickle. The tag CSV, `latent_tags.csv`, is keyed by `ts_id`. The merged feature table retains the summary features used by the rules together with the resulting tags, making it a diagnostic trace of the construction process. The prevalence table describes how often each proxy label fires; those values are descriptive properties of the labels, not clinical validation. The mortality-by-tag table reports unadjusted mortality association by proxy value, and its risk ratios are not causal estimates. The co-occurrence table reports pairwise association among proxy states. The JSON validation summary stores available-latent metadata, prevalence records, mortality-association records, sanity checks, and interpretation notes. None of these files is a substitute for expert chart-review validation.

5.4 Predicted and Majority-Vote Proxy States

The same `LAT_*` column names can appear in three different source types. A rule-derived proxy state is generated by the PhysioNet or MIMIC tagger rules above. A predicted proxy state

is exported by a supervised time-series model described in Chapter 6. A majority-vote proxy state is generated by aligning several binary voter files and applying a deterministic voting rule. These source types share an interface but not a generation mechanism.

Prediction exports contain `ts_id`, one probability column named `<label>_prob` per proxy target, and one thresholded binary column named exactly as the target label. The STraTS export code thresholds probabilities at 0.5. Downstream voting does not consume probability columns as votes. The helper `split_predicted_latent_tags.py` separates the probability half and binary-tag half of a combined prediction CSV, while the parent router can also drop `_prob` columns and write a binary-only voter table in configured `LAT_*` order. Export provenance remains incomplete, so this chapter does not claim that every archived prediction CSV is out-of-sample or tied to a verified checkpoint manifest.

The active majority-vote script discovers non-hidden `.csv` files in the input voter directory, sorts them by path string, and treats the first file as the reference column-order file. Every voter file must contain `ts_id`, have no duplicate normalized identifiers, and contain at least one non-identifier column. All non-`ts_id` columns are treated as latent columns; nonbinary values, including unsplit probability columns, cause validation failure. Later voters must have exactly the same latent-column set as the reference, although they are reordered to the reference order before voting. Rows are aligned on the intersection of `ts_id` values shared by all voters, and the output order follows the reference file restricted to that shared intersection.

For record i, proxy state k, and M aligned voter files, the implemented rule is

$$\widetilde{Z}_{ik} = \mathbb{I}\left(\sum_{m=1}^{M} Z_{ik}^{(m)} \geq \left\lceil \frac{M}{2} \right\rceil\right).$$

This is equivalent to the source expression `2 * ones_count >= n_voters`; therefore an even-voter tie is mapped to 1. The output schema is `ts_id` followed by the reference latent columns. The causal orchestrator writes this table under the run's `majority_vote/` directory and then passes it to mortality prediction, matching, CATE estimation, sensitivity analysis, and permutation stages.

Majority voting aggregates binary model or rule outputs; it does not establish clinical truth, remove shared model bias, guarantee independence among voters, or prove ensemble superiority. It can amplify correlated errors when voters are trained on related data or share the same rule-derived targets. Its interpretation depends on identical construct definitions, aligned identifiers, binary-only inputs, and the exact voter set supplied to each run. The available run summaries confirm the majority-vote stage and record absolute voter input directories, but they do not archive a per-run voter manifest listing every source file and hash.

[VALIDATION REQUIRED: create a per-run majority-vote voter manifest listing every binary voter CSV, source predictive model, export split, checkpoint hash, input-artifact hash, row count, latent ordering, and majority-vote output hash]

Chapter 6

Predictive Modeling of Proxy States

This chapter describes the predictive layer that estimates rule-derived proxy-state vectors from ICU time-series records. The supervised target for this layer is a proxy-label matrix loaded from a separate CSV file keyed by `ts_id`; the inputs are irregular measurement histories plus static covariates prepared under the data contracts described in Chapter 4. The models therefore predict analytical proxy labels rather than adjudicated clinical states.

The implementation compares STraTS-family irregular-observation models with recurrent, convolutional, attention-based, decay-based, and interpolation-based baselines. This chapter defines the representation, training, evaluation, and export mechanics needed to connect these predictors to the later proxy-aggregation workflow. Final run configurations, selected result artifacts, and numerical model comparisons are intentionally deferred to Chapters 9 and 10.

6.1 Self-Supervised STraTS Pretraining

STraTS represents an irregular time series as a set of observed measurement tokens rather than as a fully imputed grid. The implemented STraTS class constructs each token by adding three learned components: a continuous embedding of measurement time, a continuous embedding of the normalized measurement value, and an embedding of the variable identity. The token sequence is contextualized with a transformer-style block and then pooled with fusion attention to obtain a fixed-dimensional time-series representation [15]. Static covariates are combined with this representation before either the self-supervised forecasting head or the supervised proxy-label head is applied.

The same source file implements both `strats` and `istrats` model types. In the verified implementation, `strats` pools contextualized observation embeddings and passes static covariates through a learned demographic embedding. The `istrats` branch instead pools the initial triplet embeddings and concatenates the raw static-covariate vector. The pretraining command-line guard permits self-supervised pretraining only for `strats` and `istrats`; the recurrent, convolutional, attention-grid, decay, and interpolation baselines do not have a supported self-supervised pretraining path in the current source.

The pretraining dataset loader reads the split-aware processed artifact and removes held-out

test identifiers before constructing unsupervised examples. It builds the variable vocabulary from variables observed in the pretraining train split, derives value means and standard deviations from that train split, and saves the resulting variable list, normalization table, and maximum time horizon to `pt_saved_variables.pkl`. For PhysioNet 2012 the maximum time is 48 hours. For MIMIC-III the loader permits observations up to five days but samples at most a 24-hour input context and clamps implausibly old age values in the same way as the supervised loader.

Each self-supervised example is formed by sampling a forecast anchor from observed timestamps that are not the final timestamp and are at least 12 hours after record start. Observations up to the anchor form the historical context, truncated by the maximum observation count and, for MIMIC-III, by the 24-hour context limit. The target window extends two hours after the anchor. For each variable observed in that future window, the implementation keeps the last observed normalized value and sets the corresponding forecast mask entry to one; variables without future observations are masked out of the loss. Input times are rescaled to the interval $[-1, 1]$ using the dataset-specific maximum time.

The pretraining head maps the combined time-series and static representation to one forecast value per variable. Its objective is masked squared error over variables observed in the sampled future window, so unobserved future variables do not contribute to the loss. The pretraining evaluator materializes batches for each evaluation split, sampling three forecast batches per evaluation chunk when a split is first evaluated, and reports `loss_neg`, the negative masked loss, for checkpoint selection. The best pretraining checkpoint is written as `checkpoint_best.bin`; loss values themselves are not reported in this methods chapter.

Checkpoint-loaded supervised training is implemented by constructing the target supervised architecture, loading the supplied pretraining checkpoint, and copying overlapping tensors into the current model state before calling `load_state_dict`. When `--load_ckpt_path` is supplied, the supervised dataset also loads `pt_saved_variables.pkl` from the same directory to reuse the pretraining variable vocabulary, normalization statistics, and maximum-time metadata. This source path supports warm-started STraTS-family training, but the archived final exports still lack a complete manifest linking each exported CSV to the intended pretraining checkpoint, supervised checkpoint, split artifact, and archive-copy step.

[VALIDATION REQUIRED: recover a predictive manifest linking each final STraTS-family supervised checkpoint, pretraining checkpoint, export command, and exported CSV]

6.2 Supervised Multi-Label Models

The supervised prediction task uses a latent-label CSV as its target matrix. The loader accepts identifier columns named `ts_id`, `icustay_id`, or `ICUSTAY_ID`, canonicalizes them to `ts_id`, and checks consistency when more than one accepted identifier column is present. It then filters the label table to the supervised split identifiers, raises an error for missing labels, raises an error for duplicate normalized identifiers, sorts rows to match the internal supervised identifier

order, and treats every non-`ts_id` column as a proxy-label target. The number and order of supervised targets are therefore determined at runtime by the CSV schema.

All supervised backends share a multi-label output contract. During training the model returns binary cross-entropy with logits against the full target vector, using per-target positive-class weights computed from the supervised train split. During evaluation or prediction export, the same model call without labels returns sigmoid probabilities. Static covariates are normalized from training rows and are concatenated, either after a learned demographic embedding or directly in the `istrats` branch, with each model's time-series representation. In scratch supervised training the binary head maps the combined representation directly to the proxy-label logits. In checkpoint-loaded training the source inserts an intermediate forecasting head before the binary head so that overlapping pretraining tensors can be reused.

The STraTS supervised backend uses the irregular observation triplets described in Section 6.1: time, value, and variable embeddings, transformer contextualization, fusion attention, and static-feature combination [15]. The `istrats` branch shares the same triplet construction and fusion-attention module, but its verified pooling and static-feature path differ as described above. The current final artifact archive contains STraTS prediction CSVs, while an iSTraTS final prediction artifact was not found during this stage.

The GRU baseline uses a dense hourly grid rather than the raw irregular token set. For each hour and variable, the loader builds value, observation-mask, and elapsed-time-since-observation channels, mean-fills unobserved values from training data, normalizes the filled values, and concatenates the three channel groups before passing them to a gated recurrent unit [16]. The final recurrent hidden state is concatenated with the static-feature embedding and projected through the shared supervised head.

The GRU-D backend uses a model-specific irregular sequence representation containing observed values, missingness masks, elapsed-time tensors, sequence lengths, and static covariates [17]. The implemented cell updates last observed values, applies learned exponential decay to inputs and hidden states as a function of elapsed time, and includes missingness masks in the recurrent gates. The representation passed to the supervised head is the final valid recurrent state for each sequence combined with the static-feature embedding.

The TCN baseline uses the same dense hourly value, observation-mask, and elapsed-time representation as the GRU baseline, but processes the sequence with temporal convolutional residual blocks [18]. The implementation permutes the dense grid into channel-first form, applies a temporal convolutional network, takes the last temporal embedding, and concatenates it with the static-feature embedding before projection to proxy-label logits.

The SAnD backend also consumes the dense hourly representation. It applies a one-dimensional input embedding, learned positional encoding, masked attention blocks constrained by a local attention window, and dense interpolation over a fixed number of reference positions before combining the resulting time-series representation with static covariates [19]. In this repository, SAnD is therefore an attention-based dense-grid comparator rather than an irregular-token model.

InterpNet is implemented as an interpolation-prediction baseline for irregularly sampled time series [20]. The loader supplies observation times in hours, value matrices, observation masks, and holdout masks. The model performs single-channel interpolation, cross-channel interpolation, concatenates interpolation-derived quantities, and feeds the resulting sequence to a GRU before applying the shared supervised head. When labels are present, the implementation adds an auxiliary reconstruction loss over held-out observed values. Implementation presence does not establish an approved final InterpNet performance artifact; no final InterpNet prediction CSV or training summary was found in the inspected final archive.

[RESULT REQUIRED: approved final InterpNet evaluation artifact before inclusion in the predictive performance comparison]

Table 6.1: Implemented predictive model families and non-numeric evidence status.

Model	Temporal representation and irregularity handling	Static features and pretraining path	Citation and repository evidence status
STraTS	Irregular observation tokens built from time, value, and variable embeddings; observation masks handle padding, and fusion attention pools contextualized tokens.	Learned demographic embedding is concatenated with the pooled representation; self-supervised pretraining is supported.	[15]. Implementation confirmed; wrappers and archived STraTS outputs present; final checkpoint-to-export manifest incomplete.
iSTraTS	Same initial triplet construction as STraTS, but fusion attention pools initial triplet embeddings rather than contextualized embeddings.	Raw static-covariate vector is concatenated; self-supervised pretraining is supported.	[15]. Implementation confirmed; no approved final iSTraTS export found in inspected archive.
GRU	Dense hourly grid with value, observation-mask, and elapsed-time channels; unobserved values are mean-filled before normalization.	Learned demographic embedding is concatenated with the final recurrent state; pretraining is not supported by the current guard.	[16]. Implementation confirmed; archived GRU prediction CSVs present; export provenance incomplete.
GRU-D	Irregular sequence of value, mask, elapsed-time, and sequence-length tensors; learned decays and masks enter the recurrent update.	Learned demographic embedding is concatenated with the final valid recurrent state; pretraining is not supported by the current guard.	[17]. Implementation confirmed; archived GRU-D prediction CSVs present; export provenance incomplete.

Model	Temporal representation and irregularity handling	Static features and pretraining path	Citation and repository evidence status
TCN	Dense hourly grid with value, observation-mask, and elapsed-time channels processed by temporal convolutional residual blocks.	Learned demographic embedding is concatenated with the last convolutional embedding; pretraining is not supported by the current guard.	[18]. Implementation confirmed; archived TCN prediction CSVs present; export provenance incomplete.
SAnD	Dense hourly grid with input embedding, positional encoding, masked attention, and dense interpolation; mask/delta channels come from the loader.	Learned demographic embedding is concatenated after dense interpolation; pretraining is not supported by the current guard.	[19]. Implementation confirmed; archived SAnD prediction CSVs present; export provenance incomplete.
InterpNet	Irregular times, values, observation masks, and hold-out masks are transformed by single-channel and cross-channel interpolation before a GRU.	Learned demographic embedding is concatenated with the GRU representation; pretraining is not supported by the current guard.	[20]. Implementation confirmed; no approved final result artifact found in inspected archive.

The supervised evaluator computes metrics independently for each target column in the requested split. Target columns with only one observed class in that split are skipped. For the remaining targets, it computes AUROC, area under the precision-recall curve, and the maximum value of the pointwise minimum of precision and recall. The reported split-level values are simple averages over the non-degenerate target columns, with loss and negative loss also recorded. During supervised training, checkpoint selection uses the validation value of `auprc + auroc`; this validation score should not be interpreted as a test score.

Training mechanics are shared across the supervised backends at the top-level script. The script constructs a deterministic seed by adding the run index to the configured seed, uses AdamW for trainable parameters, clips gradient norms at 0.3, evaluates at the configured validation schedule, and saves `checkpoint_best.bin` when the validation selection metric improves. If an `eval_train` split is evaluated, it is the first 2000 training examples in the current supervised split object, which makes it a diagnostic subset rather than a separate external evaluation set.

6.3 Prediction Export and Normalization

Prediction export is requested with `--save_pred_csv_path`. The export routine can run after a training run or in an export-only invocation with no training steps scheduled. Immediately before export, the script reloads `checkpoint_best.bin` from the current output directory only if that file exists. The exported split is selected by `--predict_split`, whose accepted values are `train`, `val`, `test`, and `all`; in the `all` case the exporter iterates over all supervised identifiers in the train, validation, and test split object. The inspected wrapper scripts and archived export logs support `predict_split=all` for the final prediction CSVs, but they do not provide a complete processed-artifact, checkpoint, and archive-copy manifest.

For every exported row, the exporter removes labels from the batch, obtains probability outputs from the model, thresholds each probability at 0.5, and writes a combined CSV. The first column is `ts_id`. It is followed by one probability column named `<label>_prob` for each target and then one thresholded binary prediction column named exactly as the target label. These binary columns are model-predicted proxy labels, not rule-derived labels and not clinical adjudications.

Table 6.2: Prediction export schema before downstream voter normalization.

Column group	Naming pattern	Meaning	Source of value
Identifier	`ts_id`	Normalized supervised record identifier.	STraTS dataset loader and split object.
Probability outputs	`<label>_prob`	Sigmoid model probability for a proxy-label target.	Supervised model called without labels.
Binary predictions	`<label>`	Thresholded model-predicted proxy label.	Probability output thresholded at 0.5.

The combined prediction CSV is not the same file shape required by downstream voter aggregation. The helper `split_predicted_latent_tags.py` validates that `ts_id` is the first column, expects an even number of non-identifier columns, splits the first half into probability columns and the second half into binary tag columns, and writes separate probability and absolute-tag CSVs. The majority-vote input contract is stricter: each voter file must contain `ts_id` plus binary proxy columns, with all voter files sharing the same latent-column set after validation. The parent router can collect STraTS outputs, drop probability columns, enforce the approved latent ordering, and write normalized voter CSVs for later aggregation. The semantics of the majority-vote proxy label belong to Chapter 5; here the important point is the predictive handoff contract.

[VALIDATION REQUIRED: record the final export split, source processed-pickle hash, split-generation seed or ID manifest, source checkpoint, and archive-copy provenance for each prediction CSV]

Chapter 7

Causal Graph and Effect-Estimation Methodology

7.1 Dataset-Specific DAGs

The causal component of this project does not learn a causal graph from the observational data. It uses two project-specified directed acyclic graphs (DAGs), one for PhysioNet 2012 and one for MIMIC-III, as explicit design assumptions for selecting adjustment variables and organizing later effect-estimation outputs. The graph structure is informed by LLM-assisted design elicitation, project prompt artifacts, manager summaries, and source-code implementation, but the graph authority in the executable pipeline is the active graph-construction code. The diagrams should therefore be read as encoded causal hypotheses, not as clinically validated or statistically discovered ground truth.

This separation is important because the main exposure variables in the causal stage are binary proxy states rather than randomized treatments. In the causal code, each selected proxy state is treated as a binary exposure A, the outcome is in-hospital mortality Y, and the graph is used to identify candidate pre-exposure background or latent proxy variables for adjustment. The thesis follows the graphical language of Pearl's DAG and backdoor framework while retaining the cautions around observational, proxy-defined exposures and well-defined interventions discussed by Hernan and Taubman [1, 3]. These cautions are especially relevant here because several variables named as "treatments" in the code are clinically closer to patient states or care-process proxies than to assigned therapeutic interventions.

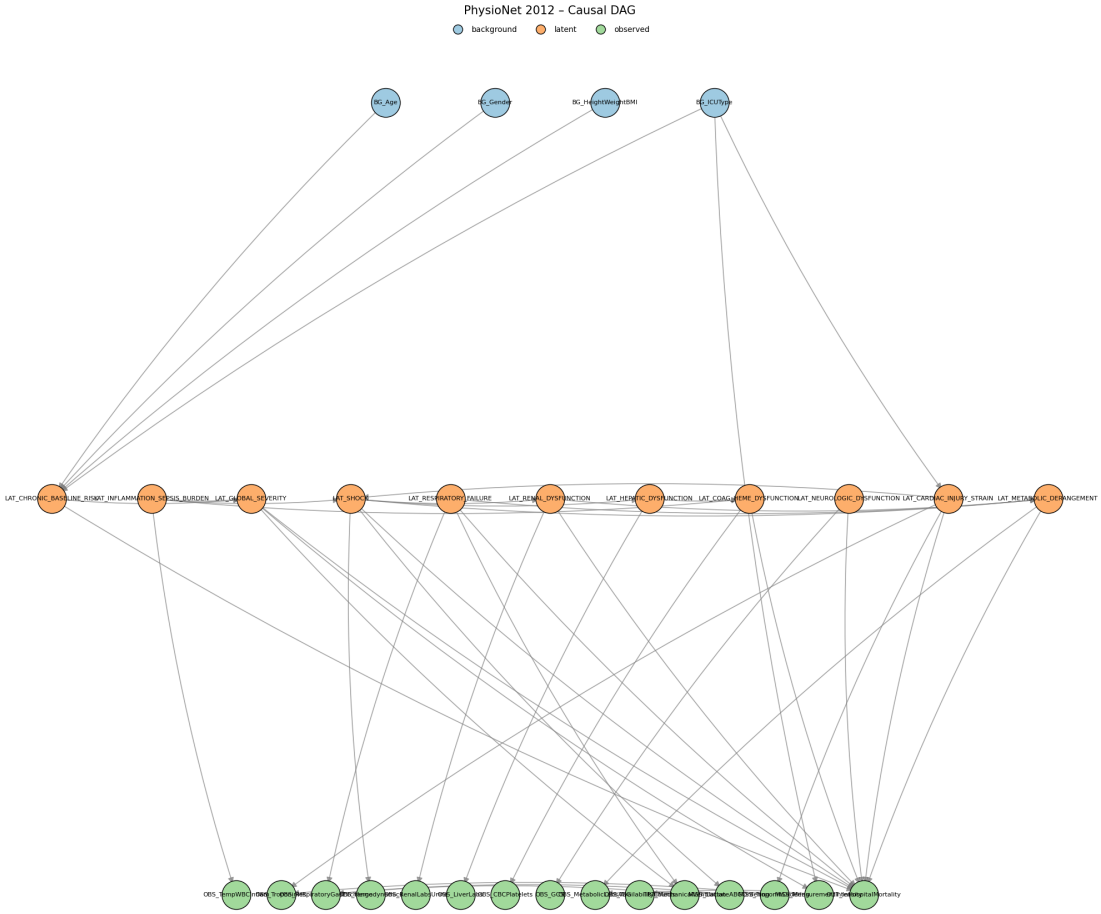

Figure 7.1: Project-specified PhysioNet 2012 DAG used by the causal pipeline. The figure is a thesis-local copy of the audited graph artifact generated by the active PhysioNet graph-construction source. Arrows encode assumed causal directions rather than statistically learned relationships; exact node and edge definitions remain source-code definitions.

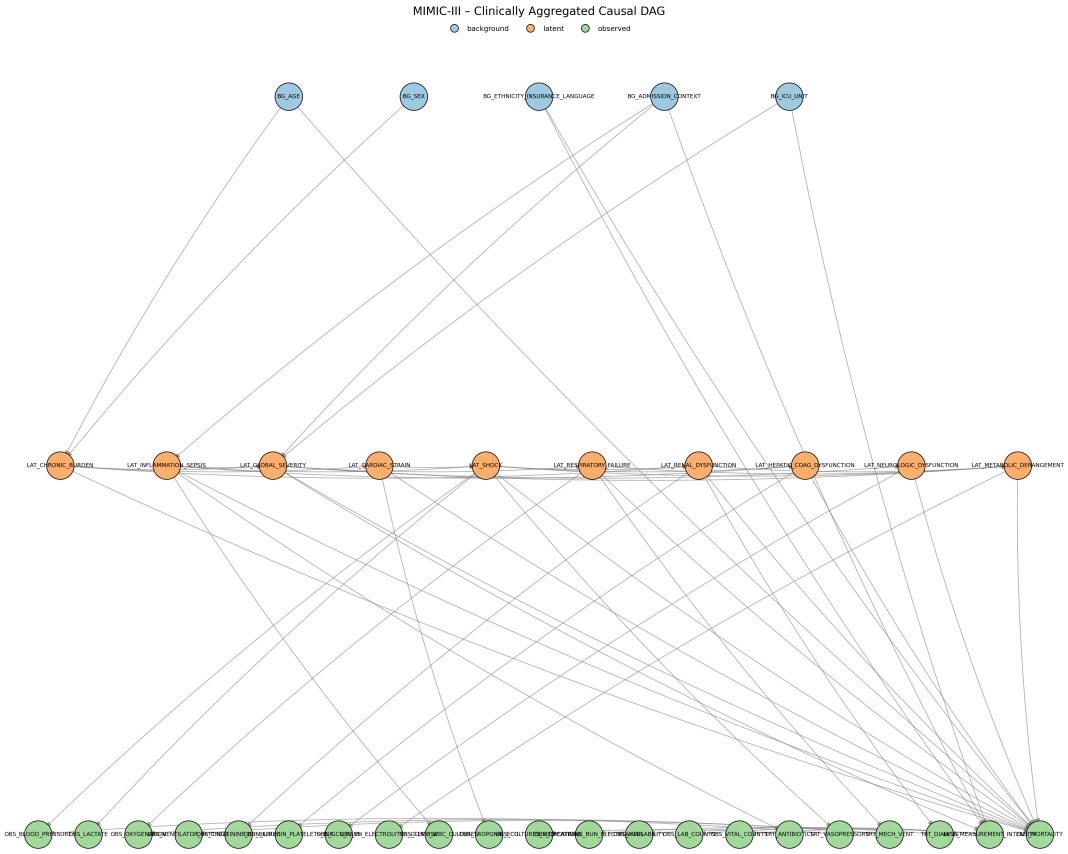

Figure 7.2: Project-specified MIMIC-III DAG used by the causal pipeline. The figure is a thesis-local copy of the audited graph artifact generated by the active MIMIC graph-construction source. Arrows encode assumed causal directions rather than statistically learned relationships; exact node and edge definitions remain source-code definitions.

Table 7.1: Implemented DAG node families used by the causal pipeline.

family	role in graph	PhysioNet 2012 implementation	MIMIC-III implementation
Background	Baseline variables allowed as adjustment candidates if mapped to dataframe columns.	Four nodes: age, gender, height/weight/BMI, and ICU type.	Five nodes: age, sex, ethnicity/insurance/language, admission context, and ICU unit.
Latent or proxy state	Binary project proxy states. Selected non-chronic states are analyzed as exposure variables; eligible pre-exposure latent nodes may also enter adjustment sets.	Eleven nodes, including chronic baseline risk, inflammation/sepsis burden, global severity, shock, respiratory failure, renal dysfunction, hepatic dysfunction, coagulation/hematologic dysfunction, neurologic dysfunction, cardiac injury/strain, and metabolic derangement.	Ten nodes, including chronic burden, inflammation/sepsis, global severity, cardiac strain, shock, respiratory failure, renal dysfunction, hepatic/coagulation dysfunction, neurologic dysfunction, and metabolic derangement.
Observed measurements	Measurement aggregates and manifestations. These document how clinical constructs relate to observed data but are excluded from adjustment-set candidates.	Ten observed aggregate nodes, including inflammatory, cardiac, respiratory, hemodynamic, renal, liver, hematologic, neurologic, metabolic, and availability-count measurements.	Fifteen observed aggregate nodes, including vital, laboratory, medication, procedure, ventilator, culture, and availability/count nodes.

family	role in graph	PhysioNet 2012 implementation	MIMIC-III implementation
Treatment or care process	Care-process nodes retained in the graph to represent downstream clinical response or measurement context. These are not treated as adjustment candidates.	Mechanical ventilation node.	Antibiotics, vasopressors, mechanical ventilation, and dialysis nodes.
Missingness or measurement intensity	Process nodes for ordering and measurement availability. These are graph variables but not adjustment candidates.	Lactate/ABG ordering, troponin ordering, and measurement intensity.	Measurement intensity, with outgoing links to availability, laboratory-count, and vital-count nodes.
Outcome	Mortality endpoint.	`OUT_InHospitalMortality`.	`OUT_MORTALITY`.

The PhysioNet graph contains 30 nodes and 45 directed edges. The implemented exposure list contains the ten non-chronic latent proxy states in the PhysioNet configuration: global severity, shock, respiratory failure, renal dysfunction, hepatic dysfunction, coagulation/hematologic dysfunction, inflammation/sepsis burden, neurologic dysfunction, cardiac injury/strain, and metabolic derangement. Chronic baseline risk is retained as an upstream baseline proxy and effect-modifier candidate rather than as an analyzed exposure.

The MIMIC graph contains 36 nodes and 57 directed edges. The implemented exposure list contains nine non-chronic latent proxy states: global severity, inflammation/sepsis, shock, respiratory failure, renal dysfunction, hepatic/coagulation dysfunction, neurologic dysfunction, cardiac strain, and metabolic derangement. Chronic burden is retained as an upstream baseline proxy and effect-modifier candidate. The MIMIC graph contains richer administrative and care-process structure than the PhysioNet graph, but the downstream adjustment implementation only uses graph nodes that can be mapped into the analysis dataframe.

[SUPERVISOR DECISION REQUIRED: approve the causal interpretation and clinical plausibility of the PhysioNet and MIMIC project DAGs]

[VALIDATION REQUIRED: document the human and clinical review decisions applied to the

LLM-assisted DAG proposals]

[VALIDATION REQUIRED: recover the exact graph source commit, config, command, and output hashes for the canonical DAG artifacts]

7.2 Adjustment-Set Logic

For each dataset, treatment-like proxy state, and outcome pair, the causal pipeline derives an adjustment set from the project DAG and the columns available in the analysis dataframe. The implementation first maps dataframe variables to graph nodes and graph nodes back to dataframe columns. Background and latent graph nodes are eligible only if they can be represented by available columns. Observed measurement nodes, care-process nodes, missingness nodes, and the outcome node are excluded from candidate adjustment sets.

Let G denote the implemented project DAG, A the binary proxy-state exposure, and Y the in-hospital mortality outcome. The code constructs a graph $G_{\underline{A}}$ by removing outgoing edges from A. This is a project helper for finding adjustment candidates; it should not be confused with the usual intervention graph notation that removes incoming edges into A. The initial candidate pool is:

$$\mathcal{C}(A, Y) = \mathcal{A}_{\text{allowed}} \cap \text{An}_G(A) \cap \text{An}_{G_{\underline{A}}}(Y) \setminus \text{De}_G(A) \setminus \{A, Y\},$$

where $\mathcal{A}_{\text{allowed}}$ is the set of available graph nodes whose node type is background or latent. The implementation then enumerates backdoor paths between A and Y in the undirected skeleton whose first directed edge points into A. A path is treated as blocked if an adjusted non-collider is present or if an unadjusted collider has no adjusted descendant. This is an encoded graphical test over the project DAG, not proof that all real-world confounding has been controlled.

The preferred helper is the safe expanded adjustment mode. It begins with the full candidate pool, removes colliders on the encoded backdoor paths and removes their descendants, then checks whether the remaining pool blocks all encoded backdoor paths. If it succeeds, this expanded pool is returned. If it does not block all paths, the pipeline falls back to a greedy minimal helper that starts from the candidate pool and removes variables in sorted order only while all encoded backdoor paths remain blocked. The resulting set is inclusion-minimal under the implementation order; it is not guaranteed to be the unique minimal adjustment set.

Table 7.2: Implemented adjustment-set logic and thesis interpretation.

implementation step	source behavior	thesis interpretation
Graph and column aliasing	Maps selected dataframe columns such as age, sex/gender, ICU type, admission context, and chronic proxy states to project graph nodes.	Adjustment is limited by what the analysis dataframe actually contains; graph nodes without available mapped columns cannot be adjusted for.
Allowed nodes	Keeps only graph nodes typed as background or latent and available in the dataframe.	Measurement, care-process, missingness, and outcome variables are deliberately excluded from candidate adjustment sets.
Candidate pool	Intersects allowed nodes with ancestors of exposure and ancestors of outcome in the outgoing-edge-removed graph, then removes exposure descendants and the exposure/outcome nodes.	Candidate variables are source-defined pre-exposure or upstream variables under the project DAG.
Backdoor-path check	Enumerates encoded backdoor paths and applies collider/non-collider blocking rules.	The check assesses the implemented graph, not unmeasured real-world mechanisms outside the DAG.
Safe expanded set	Removes colliders and descendants of colliders before testing whether the remaining pool blocks paths.	Main adjustment mode, intended to reduce collider-conditioning risk.
Minimal fallback	Greedily removes sorted candidate variables while preserving path blocking.	A deterministic implementation fallback, not a claim that no alternative sufficient set exists.
Identifiability flag	Records whether available mapped variables block all encoded backdoor paths in the project DAG.	A project-DAG availability diagnostic; it should not be described as definitive causal identification in the clinical population.

Table 7.3: Causal assumptions required for interpreting the estimator outputs.

assumption	methodological requirement	repository support	unresolved limitation
Graph correctness for the target question	The DAG must correctly represent the relevant causal structure for the exposure contrast and mortality outcome.	Dataset-specific graphs are project-specified and encoded in active source.	The arrows were not learned from data, and the human clinical review record remains incomplete.
Sufficient measured adjustment	Available adjustment variables must block the relevant open backdoor paths for each proxy exposure.	The helper tests path blocking within the implemented DAG using mapped background and latent columns.	Graph variables may be unavailable, and unmeasured or misspecified confounding outside the project DAG remains possible.
Positivity or overlap	Comparable covariate strata must contain both exposed and unexposed records.	Matching outputs report match availability and match rates.	Dedicated treatment-specific overlap diagnostics and thresholds remain to be approved.
Consistency and well-defined exposure	Each binary proxy-state contrast must have a sufficiently stable meaning across records.	The pipeline applies one binary exposure column at a time.	Disease-state and severity proxies are not necessarily assignable interventions, so the causal estimand remains unresolved.
Proxy-state measurement	The binary exposure should measure the intended construct with sufficiently limited construct and classification error.	Exposure columns are rule-derived, predicted, or majority-vote proxy states.	No code path verifies construct validity or removes proxy-state measurement error; clinical and chart-review validation remains incomplete.
Outcome measurement	The processed `in_hospital_mortality` field must be consistently and correctly constructed from the outcome artifact.	Both estimator paths merge this processed outcome column by record identifier and require it to be present.	Source checks establish schema availability, not endpoint validity or cross-dataset construction equivalence.

assumption	methodological requirement	repository support	unresolved limitation
Temporal ordering and avoidance of post-exposure adjustment	Adjustment variables must be measured or conceptually occur before the relevant proxy-state contrast; post-exposure adjustment should be avoided.	Protection depends on the project DAG, source node classification, descendant filtering, and allowed-node filtering.	Repository graph ordering does not prove patient-record timing; actual variable timing and the correctness of the proxy-state construction window remain unverified.
Nuisance- and effect-model adequacy	LinearDML and CausalForestDML require sufficiently adequate nuisance and effect models, regularity conditions, and appropriate cross-fitting behavior for their intended interpretation.	The source configures EconML estimators, random-forest nuisance models, and estimator-provided fitting behavior.	Source configuration alone does not verify model adequacy, regularity conditions, calibration, or valid cross-fitting performance in these data.
No interference	One record's proxy-state exposure should not affect another record's outcome under the working unit-level interpretation.	Estimation treats records as separate units.	The repository does not empirically test interference or dependence across admissions, patients, units, or care processes.
Sampling and target population	The analysis sample must support the stated target population, or reweighting or another transport argument must justify a sampled target.	The optional outcome-downsampling condition is deterministic given the configured seed and is applied consistently before treatment iteration.	Original and outcome-downsampled analyses correspond to different empirical populations; no weighting or target-population correction is documented.

[SUPERVISOR DECISION REQUIRED: approve the final estimand wording for proxy-state exposures]

[VALIDATION REQUIRED: validate treatment-specific adjustment sets against the selected final run artifacts]

7.3 Matching and Heterogeneous-Effect Estimators

The causal pipeline applies two families of estimators to the binary proxy-state exposures: a transparent matched-pair baseline and model-based heterogeneous-effect estimators. Both families use the same broad analysis frame. Patient-level proxy-state labels are merged with the canonical in-hospital mortality outcome and baseline covariates. For a selected exposure $A_i \in \{0, 1\}$, outcome Y_i, adjustment variables W_i, and effect modifiers X_i, the target working quantity is a contrast in predicted or matched mortality outcome between $A_i = 1$ and $A_i = 0$, conditional on the source-defined covariate information. Because the exposures are proxy states and the data are observational, these contrasts require cautious interpretation.

An optional analysis condition, controlled by `DOWN_SAMPLE`, down-samples mortality outcome classes rather than exposure groups. All outcome-positive rows are retained. When the outcome-negative class is larger and the positive class is nonempty, outcome-negative rows are sampled without replacement until their count equals the number of outcome-positive rows; the configured random seed controls both this sample and the subsequent row shuffle. This occurs once before treatment-specific adjustment selection and estimation, including matching and DML/PFN fitting. It is not propensity trimming, treatment balancing, matching, or a correction for confounding, and it does not improve causal identification by itself. The procedure changes the analyzed population and observed outcome prevalence. Original and outcome-downsampled runs therefore represent different analysis populations, and the primary or sensitivity role of the downsampled runs remains a supervisor and result-design decision.

Outcome-based downsampling can change nuisance-model estimation, the distribution of covariates and proxy exposures, patient-level fitted effects, the population over which `mean_cate` is averaged, and the availability and composition of matched pairs. It also changes the sample outcome rate and therefore affects `normalized_CATE`, whose denominator is that rate. Effect summaries from original and outcome-downsampled analyses should not be compared as though they estimated the same population quantity without an approved sampling and estimand argument. Outcome-based case-control sampling does not necessarily invalidate every estimator, but interpretation here requires estimator-specific methodological justification and an explicit target-population argument.

The matching estimator is a deterministic baseline. For each proxy exposure, the pipeline drops rows with missing exposure or outcome, coerces the exposure and outcome to binary variables, imputes numeric adjustment variables with medians, and converts non-binary numeric adjustment variables into binary columns using sample-median thresholds. Already-binary variables, including ICU-type indicators, remain binary. The matcher then greedily pairs exposed and unexposed patients by Hamming distance over these binary adjustment columns, increasing the allowed distance up to the configured maximum. By default, controls are not reused. The per-pair contrast is

$$\Delta_j = Y_{j,A=1} - Y_{j,A=0},$$

and the matching summary reports the mean and standard deviation of these matched-pair differences, the number of matched pairs, the match rate, the final allowed Hamming distance, and the observed adjustment columns. This output is useful as a transparent matched descriptive contrast. It should not be labeled as an average treatment effect unless the estimand, graph assumptions, overlap, and matching design are later approved for that interpretation.

For model-based heterogeneous effects, the pipeline includes EconML double-machine-learning estimators and an exploratory CausalPFN branch. Double machine learning estimates nuisance outcome and treatment models and then estimates treatment effects after residualization, reducing sensitivity to nuisance-model overfitting under appropriate assumptions [21, 22]. The LinearDML branch uses random-forest nuisance models with a linear final effect model. The CausalForestDML branch uses the same random-forest nuisance-model family and a causal forest final stage, drawing on generalized random-forest and causal-forest ideas for heterogeneous treatment effects [23, 24]. This modeling choice is consistent with the broader use of machine learning for individualized treatment-effect estimation in electronic health record and critical-care settings, while still requiring domain-specific validation [25, 26, 27, 28, 29].

The CATE pipeline saves patient-level CATE values, normalized CATE values, optional 95 percent effect-interval columns when the estimator provides them, feature-importance or coefficient diagnostics when available, model artifacts, and run-level summaries. In the current source, `mean_cate` is the arithmetic mean of patient-level CATE estimates over the model rows. It is an aggregate of the fitted model's conditional estimates and should not automatically be described as the study ATE. The normalized CATE column is computed by dividing CATE by the sample outcome rate when that rate is positive. It is an implementation-defined rescaling for comparison within this project; it is not a risk ratio, relative risk, or percent reduction.

The effect-modifier matrix X is selected from configured effect-modifier columns, typically baseline variables plus chronic and inflammation/sepsis proxy states when present. The adjustment matrix W is the observed confounder set selected from the DAG helper. The implementation can allow overlap between X and W, because configured effect modifiers are not automatically removed from the adjustment set. This is a source-supported behavior, but its methodological interpretation should be reviewed before final claims are made.

Table 7.4: Implemented causal estimators and interpretation limits.

method	implementation summary	main outputs	interpretation boundary
Greedy Hamming matching	Binary adjustment matrix, median thresholding for non-binary covariates, median imputation, nearest available control within allowed Hamming distance, no default control reuse.	Matched pairs, pair effects, match rate, final distance, and per-treatment/global summaries.	Matched descriptive contrast; not automatically ATE or ATT.
LinearDML	EconML `LinearDML` with random-forest nuisance outcome and treatment models, binary treatment handling, configured W and X.	Patient-level CATE, optional intervals, coefficients when aligned, model artifact, global summaries.	Model-based heterogeneity estimate under project DAG, nuisance-model, overlap, and proxy-exposure assumptions.
CausalForest DML	EconML `CausalForestDML` with random-forest nuisance models and a causal-forest final stage.	Patient-level CATE, optional intervals, feature importance, model artifact, global summaries.	Flexible heterogeneity estimate; feature importance is a model diagnostic, not mechanism proof.
CausalPFN	Exploratory branch using deduplicated confounder and effect-modifier features. It writes CATE CSVs and metadata but skips EconML-specific sensitivity and permutation stages.	Patient-level CATE and normalized CATE; interval columns are unavailable and stored as missing.	Secondary/exploratory unless a validated primary citation and additional uncertainty/sensitivity support are added.

[SUPERVISOR DECISION REQUIRED: review the use of overlapping variables in W and X]

[SUPERVISOR DECISION REQUIRED: approve or replace the interpretation of normalized_CATE]

[SUPERVISOR DECISION REQUIRED: determine the methodological role and target-population

interpretation of the original and outcome-downsampled causal analyses]

[CITATION REQUIRED: validated primary source for the CausalPFN method before treating it as a main thesis estimator]

Chapter 8

Robustness, Sensitivity, and Validation Design

This chapter records the diagnostic framework surrounding the causal analyses in Chapter 7. It concerns empirical support, omitted-confounding sensitivity, estimator-diagnostic availability, benchmark-confounder comparisons, permutation checks, and run integrity. These procedures supplement rather than replace the project DAG, adjustment assumptions, and exposure definition. In particular, they do not validate the DAG, prove exchangeability, establish that a proxy-state exposure is well defined, or establish clinical actionability. They also do not compensate automatically for measurement error, selection bias, unmeasured confounding, insufficient overlap, an incorrect target-population interpretation, or incomplete run provenance.

8.1 Overlap and Support Diagnostics

The matching implementation provides indirect empirical support evidence under its implemented matching representation. For each proxy-state exposure, the matching summary records the number of treated and control records, number of matched pairs, match rate, final allowed Hamming distance, a sufficient-pair flag, a maximum-distance failure flag, and treatment-level failure information where matching cannot proceed. These fields are useful support indicators because a comparison requiring no or few acceptable pairs merits caution. They are not a propensity-score overlap analysis.

Matching is greedy, one-to-one, and based on a binary representation of the selected confounders. Binary columns are retained and other numeric columns may be median-binarized before Hamming distances are evaluated. Consequently, a larger accepted distance denotes weaker exact similarity in that representation; it is not a calibrated distance in the original covariate space. Failure to obtain enough pairs is a support warning, not a zero effect. Conversely, an adequate matching rate does not prove positivity for the intended target population. The binarization, greedy ordering, allowed-distance rule, and outcome-downsampled analysis frame all affect this diagnostic.

Dedicated propensity-score distributions, common-support plots, covariate balance before

and after matching, standardized mean differences, weighting effective sample sizes, treatment-probability histograms, and formal positivity diagnostics were not found in the approved archive. Not every analysis necessarily requires every one of these diagnostics, but their absence limits the strength of overlap claims. Limited overlap is a general concern for observational effect estimation, not evidence that this repository satisfies positivity [30].

Table 8.1: Planned overlap and support diagnostic design; this table contains no selected numerical results.

Diagnostic	Repository source	What it assesses	What it does not establish	Result status
Treated/control counts, pairs, and match rate	`matching/global_summary.csv`; per-treatment summaries	Whether the implemented matcher found comparison pairs	Propensity overlap, covariate balance, or positivity	archived; indirect only
Final Hamming distance and sufficient-pair flags	`matching_causal_effect.py`; matching summaries	Similarity in the matching representation under the binary and support warnings	Similarity in the original covariate space or a target-population guarantee	implemented; archived
Matching failure reason	Per-treatment summaries; cross-run matching-failure table	Why a matching contrast was not produced	A zero effect or absence of an exposure effect	archived
Propensity/common-support and balance diagnostics	Approved archive search	Dedicated treatment-probability and pre/post-match balance evidence	Any conclusion about their value in a particular run	not found
Selected thesis support display	Stage 4.6A manifest and result-selection review	Which verified diagnostic is appropriate for presentation	Empirical positivity without a dedicated diagnostic	requires Stage 4.6A selection

[RESULT-SELECTION REQUIRED: select the thesis-primary overlap and support diagnostics after the Stage 4.6A manifest audit]

[FIGURE REQUIRED: generate or recover a dedicated overlap/common-support diagnostic before claiming empirical positivity]

8.2 Sensitivity and Robustness Values

The repository separates several evidence layers that should not be collapsed into the statement that an estimator "produced sensitivity results": (1) estimator-method availability recorded during fitting; (2) values called directly from the fitted estimator during training; (3) serialized estimator parameters or residuals; (4) later saved-artifact analysis; (5) a fallback or reconstructed diagnostic; (6) benchmark-confounder comparison; and (7) a rendered sensitivity contour. Robustness-value and omitted-variable-bias sensitivity concepts provide a useful framework for this work [31, 32], but these references do not establish the correctness of local artifacts or assumptions.

During non-PFN fitting, `cate_estimation.py` records whether `robustness_value`, `sensitivity_inte` `sensitivity_summary`, `summary`, and residual access are available. It records direct-call values, source labels, unavailable-API errors, a serializable sensitivity-parameter snapshot when exposed, residual availability, estimator metadata, artifact schema version, and package/environment fields. The per-treatment model artifact and the run-level control-message CSV preserve these fields. A non-null value must therefore be interpreted together with its recorded source and status; it is not enough to inspect a numerical cell alone. EconML is the relevant software context for the DML and causal-forest branches, distinct from a claim that the software validates local identification [22].

The saved-artifact analysis requires a CATE result directory, then loads available model artifacts and reconstructs the relevant latent-tag and processed-data inputs. It first considers saved training-direct information and loaded-estimator calls, and it records residual-dependent calculations, benchmark-variable availability, per-treatment reports/CSVs, contours, warnings, and run-level aggregation. A report can be partial or failed; archived stage completion alone does not imply complete sensitivity information for every treatment.

For clarity, this thesis uses *estimator-native* for a method called on the fitted estimator, *saved-training direct* for that value preserved by the training artifact, and *saved-artifact recomputed* for a later direct call or reconstruction. A *fallback diagnostic* is a separately labelled calculation using serialized or residual-space information when the desired estimator API or serialized object is unavailable. *Unavailable*, *partial*, *failed*, and *not applicable* are status outcomes, not numerical sensitivity values. A fallback exists to retain diagnostic information across version and serialization limitations, but it depends on its own reconstruction assumptions and cannot be presented as identical to an estimator-native calculation. The Results chapter must report the source/status field alongside every selected sensitivity quantity.

Benchmark-confounder analysis seeks an interpretable scale: it compares a hypothetical omitted-confounding strength with an observed candidate covariate benchmark. The analysis checks candidate-variable availability, records the selected benchmark and source when possible, and records missing or unimplemented benchmark paths rather than treating them as null evidence. Such a comparison can expose sensitivity; it cannot show that an observed benchmark bounds every possible unmeasured confounder.

The CausalPFN branch does not implement the same EconML residual, interval, sensitivity,

or permutation workflow. Its missing diagnostics are therefore not zero sensitivity or robustness. The citation and validation limitation for CausalPFN stated in Chapter 7 remains in force.

Table 8.2: Sensitivity-diagnostic design and interpretation boundary.

Diagnostic type	Source stage	Applicable estimators	Required saved object	Output form	Interpretation boundary
Method availability	CATE fitting	EconML branches	Fitted estimator metadata	Boolean availability and errors	API availability is not a diagnostic value
Saved-training direct	CATE fitting	EconML branches where callable	Schema-versioned model artifact	Value plus source/error fields	Interpret only as labelled direct training-time evidence
Saved-artifact recomputed	Saved-CATE analysis	Non-PFN when artifact/environment permits	Model artifact and reconstructed inputs	CSV/report source fields	Later recomputation can differ from training-time access
Fallback diagnostic	Saved-CATE analysis	Non-PFN when direct paths fail	Serialized parameters or residual information	Explicit fallback source and warnings	Not identical to estimator-native output
Benchmark-confounder comparison	Saved-CATE analysis	Non-PFN where candidates are available	Candidate covariate and analysis inputs	Benchmark CSV/report	Gives a scale, not a bound on all unmeasured confounding

Diagnostic type	Source stage	Applicable estimators	Required saved object	Output form	Interpretation boundary
Sensitivity contour	Saved-CATE analysis	Non-PFN where parameters permit	Parameters/residual reconstruction and plot path	Rendered diagnostic figure	Requires source and status validation before selection
CausalPFN diagnostic status	Orchestration	CausalPFN	N/A for EconML workflow	Skipped/not applicable status	Absence is not a zero diagnostic

[RESULT-SELECTION REQUIRED: identify the thesis-primary estimator, dataset, sampling condition, exposure, and source status for any sensitivity contour]

[VALIDATION REQUIRED: distinguish estimator-native, saved-training, recomputed, and fallback sensitivity outputs for every selected treatment]

8.3 Permutation Checks and Reproducibility Validation

The permutation script is a sanity-diagnostic wrapper around repeated CATE estimation. For a treatment permutation, it copies the latent-tag dataframe and shuffles only the currently iterated treatment column. The remaining latent columns, processed outcome data, graph input, dataset configuration, and estimator type are held fixed. Each temporary latent CSV is passed to `cate_estimation.py` in a subprocess, and the temporary directory is removed after the aggregate metrics are extracted.

For an outcome permutation, the script copies the processed dataset structure and shuffles only the configured mortality outcome column in the outcome dataframe. The time-series object, identifiers, latent-tag dataframe, graph input, dataset configuration, and estimator type remain unchanged. One temporary outcome-pickle run yields treatment-level summaries for that trial, after which the temporary input and output directory are removed. Both procedures derive deterministic trial seeds from a base seed, an experiment code, the treatment index where applicable, and the trial index. Trial count, base seed, dataset, estimator, sampling condition, aggregation fields, warnings, and subprocess status must be verified for every selected archived run. The archived non-PFN files expose trial and seed fields, but result selection remains deferred.

The script captures nonzero subprocess return codes and summary-schema failures as errors rather than silently using incomplete trial data. It supports the LinearDML and CausalForest paths; CausalPFN is excluded by the orchestrator from saved-CATE analysis and permutations. The resulting aggregate CSVs are diagnostic artifacts, not final effect tables. They can indicate

whether similarly structured outputs arise after a key relationship is disrupted. They do not prove that an original effect is causal, that the DAG is correct, that confounding is controlled, that the estimator is calibrated, that the proxy exposure is clinically valid, or that an original result is unbiased. In this limited sense they are negative-control-like disruptions or null-reference exercises, not formal randomization tests. A model–identify–estimate–refute workflow motivates such checks at a general level, but does not show that this repository implements DoWhy [33].

Configuration validation is a separate reproducibility control. The Python and shell validators check the compact dataset-config schema and invoke active scripts with `--validate-config-only`; router tests additionally exercise command parsing and construction. The causal orchestrator writes `run_summary.json` with resolved stage commands, paths, statuses, timestamps, and expected outputs, while CATE and saved-analysis control-message CSVs record field-level provenance. These are different evidence layers: schema/config validation, command-construction validation, stage execution status, artifact existence, and scientific validity. A successful config check does not prove full execution, and a successful stage status does not validate a causal claim.

Table 8.3: Permutation and reproducibility-validation design.

Check	Disrupted element	Held-fixed elements	Output	Primary purpose	Limitation
Treatment permutation	One iterated treatment column in copied latent tags	Other latent columns, outcome data, graph, config, estimator	Aggregate treatment-permutation CSV	Sanity check after exposure-label disruption	Does not establish identification or calibration
Outcome permutation	Mortality column in copied processed outcome dataframe	Time series, IDs, latent tags, graph, config, estimator	Aggregate outcome-permutation CSV	Null-reference exercise after outcome disruption	Does not validate proxy exposure or DAG
Configuration validation	None; configuration only	Compact schema and declared script contracts	PASS/FAIL and resolved-field output	Detect invalid/missing config fields	Does not load data or execute analysis

Check	Disrupted element	Held-fixed elements	Output	Primary purpose	Limitation
Run-summary stage status	None; archived orchestration record	Stage command, log, expected outputs	`run_summary.json`	Execution/provenance audit	Status is not scientific validity
Artifact-schema check	None; header/status inspection	Selected artifact path and producing-stage context	Manifest validation record	Prevent wrong-file or wrong-granularity use	Does not establish result correctness

[VALIDATION REQUIRED: verify archived permutation trial counts, seeds, estimator type, and subprocess status for every selected run]

[SUPERVISOR DECISION REQUIRED: determine whether permutation checks are main-text evidence or appendix diagnostics]

Chapter 9

Experimental Design

This chapter defines the experiment families and reproducibility boundary used for later result selection. It covers datasets, prediction experiments, causal run families, original and outcome-downsampled conditions, configuration resolution, stage orchestration, software and hardware recording, and artifact requirements. Chapter 4 describes data preparation, Chapter 5 describes proxy-state construction, Chapter 6 describes predictive models, Chapter 7 describes causal estimators, and Chapter 8 describes diagnostic design. This chapter reports neither empirical findings nor a selected primary analysis.

9.1 Prediction Experiment Matrix

The planned predictive matrix spans PhysioNet 2012 and MIMIC-III, with STraTS, GRU, GRU-D, TCN, SAnD, and InterpNet implementation families. The archive provides supervised training summaries and prediction exports for the first five families across the two datasets; STraTS also has pretraining support. InterpNet implementation and wrapper commands are available, but approved final InterpNet training and prediction artifacts were not found. Thus, implementation availability, wrapper availability, training-artifact availability, prediction-export availability, final-result availability, and provenance completeness are separate labels rather than interchangeable evidence.

The task is multi-label prediction of dataset-specific proxy-state labels. Rule-derived and majority-vote sources provide target inputs, while STraTS artifacts use a split-aware dataset contract and prediction exports provide patient-level probability and binary-tag columns. The experiment design includes pretraining where supported, supervised fine-tuning, family-specific models, a multi-label loss with class weighting where configured, validation/test evaluation, checkpoint selection, prediction export, random-seed or split settings, and training-fraction controls. These design elements are source-confirmed; they do not by themselves recover the final producing run.

The crucial provenance gaps are processed-pickle hashes, split manifests, checkpoint-to-export mappings, exact producing wrappers/commands, archive-copy history, and the absence of final InterpNet artifacts. A requirements file describes an intended software environment,

not proof of the environment that produced an archived training summary or export.

Table 9.1: Predictive experimental matrix and artifact-status design; no predictive metric is reported.

Dataset	Model family	Pretraining support	Input representation	Primary output	Final-artifact status	Provenance limitation
PhysioNet 2012 and MIMIC-III	STraTS	Available for STraTS family	Split-aware irregular time series and proxy-label targets	Multi-label predictions and exported proxy-state columns	Training summary and export archived	Split, checkpoint, command, and copy lineage incomplete
PhysioNet 2012 and MIMIC-III	GRU, GRU-D, TCN, SAnD	Not the STraTS pretraining path	Dataset-specific irregular-series representation and proxy-label targets	Multi-label predictions and exports	Training summaries and exports archived	Final producing configuration and split/checkpoint linkage incomplete
PhysioNet 2012 and MIMIC-III	InterpNet	Wrapper/implementation availability recorded separately	Family-specific interpolation-prediction representation	Intended multi-label prediction export	Final artifact not found	No approved final training or export artifact

[VALIDATION REQUIRED: create the final predictive and causal artifact manifest with hashes and producing commands]

9.2 Causal Experiment Matrix

The causal archive contains a verified design matrix spanning PhysioNet 2012 and MIMIC-III; CausalForestDML, LinearDML, and CausalPFN; and original and outcome-downsampled conditions. The twelve archived run families are verified from `experiment_inventory.csv` and each `run_summary.json`, rather than inferred from directory names alone. Run summaries support execution-oriented metadata and stage statuses, but their referenced numbered configuration CSVs are external absolute paths and are not locally archived.

The orchestrator orders the stages as graph construction, majority vote, mortality prediction, matching, CATE estimation, saved-CATE analysis, and permutation tests. It assigns stage-specific output directories and logs, writes commands and status fields to `run_summary.json`, verifies required outputs, and records enabled, failed, or skipped stages. The CausalPFN run summaries record the saved-CATE analysis and permutation stages as skipped because that branch does not use the EconML diagnostic workflow; this is an estimator-specific limitation rather than an execution failure.

Outcome downsampling operates on mortality outcome classes, not exposure groups. It changes the empirical analysis population and sample outcome rate before treatment-specific matching and CATE estimation, and can affect matching availability, nuisance fitting, CATE aggregation, and normalized CATE. Original and outcome-downsampled runs therefore should not be treated as estimates of the same population quantity without an approved sampling, weighting, or transport argument. Neither condition is selected as primary here.

Within each causal run, the exposure loop evaluates multiple dataset-specific binary proxy-state columns. Inclusion of any exposure in a later Results chapter requires treatment-column availability, graph-node availability, valid binary values, adjustment and support diagnostics, artifact completeness, and approved estimand wording. The exposure list is a design dimension, not a list of approved causal claims.

Table 9.2: Causal experimental matrix. The table records run-family design, not effect estimates.

Dataset	Estimator	Sampling condition	Graph	Exposure source	Diagnostics expected	Archived run status	Configuration provenance
PhysioNet 2012 and MIMIC-III	CausalForestDML	Original cohort	Dataset-specific project graph	Majority-vote binary proxy-state columns	Matching, saved-CATE analysis, and permutations	Archived run family with stage records	Numbered producing config path recorded; local file missing
PhysioNet 2012 and MIMIC-III	LinearDML	Original cohort	Dataset-specific project graph	Majority-vote binary proxy-state columns	Matching, saved-CATE analysis, and permutations	Archived run family with stage records	Numbered producing config path recorded; local file missing

Dataset	Estimator	Sampling condition	Graph	Exposure source	Diagnostics expected	Archived run status	Configuration provenance
PhysioNet 2012 and MIMIC-III	CausalPFN	Original cohort	Dataset-specific project graph	Majority-vote binary proxy-state columns	Matching; CATE; EconML diagnostics not applicable	Archived run family; later diagnostic stages skipped	Numbered producing config path recorded; local file missing
PhysioNet 2012 and MIMIC-III	CausalForestDML, LinearDML, CausalPFN	Outcome-downsampled cohort	Dataset-specific project graph	Majority-vote binary proxy-state columns	Same family-specific checks, interpreted for a different empirical population	Archived run families with stage records	Numbered producing config path recorded; downsampling setting needs manifest confirmation

[SUPERVISOR DECISION REQUIRED: select the primary causal estimator and sampling condition]

[SUPERVISOR DECISION REQUIRED: determine the role of outcome-downsampled analyses]

9.3 Computational Environment and Reproducibility

Reproducibility requires separating committed source state from active worktree state. A complete record should identify the parent CliniCause commit, causal nested-repository gitlink, active causal nested-worktree status, and STraTS nested-repository state. A dirty local worktree is not reproducible from a commit alone. Similarly, run summaries record historical absolute producing-machine paths; these provide context but are not portable execution instructions, and their local existence cannot be assumed.

The active CATE code is designed to capture Python and package versions for EconML, scikit-learn, NumPy, Pandas, SciPy, Matplotlib, NetworkX, Optuna, and PyTorch; platform;

training timestamp; estimator module/class; artifact schema version; CUDA availability; and device metadata. These fields must be classified as captured by active code, actually present in an archived artifact, missing from an older run, or inferred only from a requirements file. Requirements express an intended environment, not producing-environment proof. CPU, RAM, GPU, operating-system, and cluster-node information must be reported only where stored or separately documented; paths such as `/truenas/` are not hardware evidence.

Configuration resolution follows a hierarchy of source defaults, dataset configuration CSV, command-line overrides, resolved runtime values, run summary, and saved-artifact metadata. The precedence is script-specific: for example, CATE exposes command-line overrides for selected options, whereas matching reads its downsampling setting from the dataset configuration. The numbered final-run configurations referenced by archived summaries are unavailable locally, so recovered defaults cannot be represented as proof of exact producing settings.

A complete artifact manifest should identify artifact path and role, dataset, model or estimator, sampling condition, source commit, configuration path and hash, command, input/output hashes, row/column schema, timestamp, environment, archive-copy history, and status. Constructing or validating that manifest is explicitly deferred to Stage 4.6A.

Table 9.3: Reproducibility-status design.

Reproducibility element	Available evidence	Missing evidence	Impact	Required resolution stage
Source commits	Parent history, gitlink, and nested repository state can be recorded	Exact producing commit for every archived artifact and dirty-worktree reconciliation	Commit alone may not reproduce an uncommitted producer state	Stage 4.6A
Dataset configs	Local compact defaults and archived config paths	Numbered final-run config CSVs and hashes	Exact archived settings remain configuration-incomplete	Stage 4.6A
Raw data and processed artifacts	Code contracts and path references	Access record, hashes, and processing manifest	Cohort lineage cannot be reproduced from the archive alone	Stage 4.6A / data manifest

Reproducibility element	Available evidence	Missing evidence	Impact	Required resolution stage
Predictive splits and checkpoint/export mapping	Training summaries, exports, wrappers	Split IDs/seeds, checkpoint hashes, producing commands, copy record	Final prediction provenance incomplete	Stage 4.6A
Causal run configuration and artifacts	Run summaries, logs, stage folders, schemas	Resolved configs, input/output hashes, copy history	Execution-supported but provenance incomplete	Stage 4.6A
Software environment	Active metadata collection and intended requirements	Producing environment for older final runs where absent	Version compatibility cannot be assumed	Stage 4.6A
Hardware environment	Limited device fields where archived	CPU/RAM/GPU/node and OS details where absent	Hardware claims must remain incomplete	Stage 4.6A
Result checksums and prompt/clinical review	Some source and prompt records	Final manifest hashes, prompt-run settings, clinical-review record	Review and artifact lineage remain incomplete	Stage 4.6A / supervisor review

[VALIDATION REQUIRED: recover the numbered final-run configuration CSVs or document that they are irrecoverable]

[VALIDATION REQUIRED: document the producing hardware and software environment for final runs]

9.4 Evaluation and Result-Selection Policy

Stage 4.6A must apply an explicit admission policy before numerical drafting. A candidate result is admissible only when the producing run is identified; dataset, estimator/model, and sampling condition are known; the source artifact exists; the required schema is validated; status is not failed; source fields are interpreted correctly; input/output relationships are documented; fallback diagnostics are labelled as fallback; the relevant estimand and population are stated;

and unresolved limitations remain visible. Directory names, copied summaries, or non-null cells alone are insufficient.

Table 9.4: Result-admission policy for the later manifest audit.

Item	Admission requirement	Evidence to validate
1–3	Identify producing run; identify dataset/model/estimator/sampling condition; locate source artifact	Run summary, manifest, and source path
4–6	Validate schema; exclude failed status; interpret source/status fields	Header/schema record, control CSV, report/log
7–9	Document input/output relationship; label fallback; state estimand and population	Producing command/config, artifact metadata, methods approval
10	Keep unresolved limitations visible	Manifest audit and deferred-fix/placeholder review

[VALIDATION REQUIRED: create the final predictive and causal artifact manifest with hashes and producing commands]

Chapter 10

Results

10.1 Data and Cohort Summary

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.2 Proxy Prevalence and Co-Occurrence

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.3 Predictive Performance

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.4 Learning Curves

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.5 Mortality Prediction From Proxy States

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.6 Matching Results

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.7 CATE Estimates

[RESULT REQUIRED]

[SUPERVISOR DECISION REQUIRED]

[VALIDATION REQUIRED]

10.8 Heterogeneity Diagnostics

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.9 Overlap and Support

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.10 Sensitivity

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.11 Permutation

[RESULT REQUIRED]

[VALIDATION REQUIRED]

10.12 Cross-Dataset Comparison

[RESULT REQUIRED]

[SUPERVISOR DECISION REQUIRED]

[VALIDATION REQUIRED]

Chapter 11

Discussion

11.1 Answer Research Questions

[STAGE 4 DRAFT REQUIRED]

[RESULT REQUIRED]

[VALIDATION REQUIRED]

11.2 Limitations and Threats to Validity

[STAGE 4 DRAFT REQUIRED]

[RESULT REQUIRED]

[VALIDATION REQUIRED]

Chapter 12

Conclusions and Future Work

12.1 Contribution Summary and Future Work

[STAGE 4 DRAFT REQUIRED]

[RESULT REQUIRED]

Appendix A

Complete Proxy-State Definitions

[STAGE 4 DRAFT REQUIRED]

[VALIDATION REQUIRED]

Appendix B

DAG Node and Edge Inventory

[STAGE 4 DRAFT REQUIRED]

[SUPERVISOR DECISION REQUIRED]

Appendix C

Configuration Fields and Resolved Run Settings

[STAGE 4 DRAFT REQUIRED]

[VALIDATION REQUIRED]

Appendix D

Model Hyperparameters and Training Commands

[STAGE 4 DRAFT REQUIRED]

[VALIDATION REQUIRED]

Appendix E

Additional Figures and Tables

[STAGE 4 DRAFT REQUIRED]

[FIGURE REQUIRED]

[TABLE REQUIRED]

[VALIDATION REQUIRED]

Appendix F

Reproducibility and Environment Notes

[STAGE 4 DRAFT REQUIRED]

[VALIDATION REQUIRED]

Appendix G

Legacy Code and Excluded Artifacts

[STAGE 4 DRAFT REQUIRED]

[VALIDATION REQUIRED]

Bibliography

[1] Judea Pearl. "Causal Diagrams for Empirical Research." In: *Biometrika* 82.4 (1995), pp. 669–688. DOI: 10.1093/biomet/82.4.669. URL: https://doi.org/10.1093/biomet/82.4.669.

[2] Miguel A. Hernan and James M. Robins. "Using Big Data to Emulate a Target Trial When a Randomized Trial Is Not Available." In: *American Journal of Epidemiology* 183.8 (2016), pp. 758–764. DOI: 10.1093/aje/kwv254. URL: https://doi.org/10.1093/aje/kwv254.

[3] Miguel A. Hernan and Sarah L. Taubman. "Does Obesity Shorten Life? The Importance of Well-Defined Interventions to Answer Causal Questions." In: *International Journal of Obesity* 32.Suppl 3 (2008). Author/academic repost used because publisher PDF was not directly downloadable in this environment., S8–S14. DOI: 10.1038/ijo.2008.82. URL: https://doi.org/10.1038/ijo.2008.82.

[4] Ikaro Silva et al. "Predicting In-Hospital Mortality of ICU Patients: The PhysioNet/Computing in Cardiology Challenge 2012." In: *Computing in Cardiology*. Vol. 39. 2012, pp. 245–248. URL: https://physionet.org/content/challenge-2012/1.0.0/.

[5] Alistair E. W. Johnson et al. "MIMIC-III, a freely accessible critical care database." In: *Scientific Data* 3.160035 (2016). DOI: 10.1038/sdata.2016.35. URL: https://doi.org/10.1038/sdata.2016.35.

[6] Hrayr Harutyunyan et al. "Multitask Learning and Benchmarking with Clinical Time Series Data." In: *Scientific Data* 6.1 (2019), p. 96. DOI: 10.1038/s41597-019-0103-9. URL: https://doi.org/10.1038/s41597-019-0103-9.

[7] Juan M. Banda et al. "Advances in Electronic Phenotyping: From Rule-Based Definitions to Machine Learning Models." In: *Annual Review of Biomedical Data Science* 1 (2018), pp. 53–68. DOI: 10.1146/annurev-biodatasci-080917-013315. URL: https://doi.org/10.1146/annurev-biodatasci-080917-013315.

[8] Alexander Ratner et al. "Snorkel: Rapid Training Data Creation with Weak Supervision." In: *The VLDB Journal* 29 (2020), pp. 709–730. DOI: 10.1007/s00778-019-00552-1. URL: https://doi.org/10.1007/s00778-019-00552-1.

[9] Mervyn Singer et al. "The Third International Consensus Definitions for Sepsis and Septic Shock (Sepsis-3)." In: *JAMA* 315.8 (2016), pp. 801–810. DOI: 10.1001/jama.2016.0287. URL: https://doi.org/10.1001/jama.2016.0287.

[10] Jean-Louis Vincent et al. “The SOFA (Sepsis-related Organ Failure Assessment) Score to Describe Organ Dysfunction/Failure.” In: *Intensive Care Medicine* 22.7 (1996). No lawful public PDF was downloaded: Springer/Ovid routes returned HTML rather than a PDF; metadata retained with missing PDF status., pp. 707–710. DOI: 10.1007/BF01709751. URL: https://doi.org/10.1007/BF01709751.

[11] Patrick Essay, Jarrod Mosier, and Vignesh Subbian. “Rule-Based Cohort Definitions for Acute Respiratory Failure: Electronic Phenotyping Algorithm.” In: *JMIR Medical Informatics* 8.4 (2020), e18402. DOI: 10.2196/18402. URL: https://doi.org/10.2196/18402.

[12] ARDS Definition Task Force et al. “Acute Respiratory Distress Syndrome: The Berlin Definition.” In: *JAMA* 307.23 (2012). No lawful public PDF was downloaded: JAMA/CDN routes returned 403 and Ovid returned HTML rather than a PDF; metadata retained with missing PDF status., pp. 2526–2533. DOI: 10.1001/jama.2012.5669. URL: https://doi.org/10.1001/jama.2012.5669.

[13] Kidney Disease: Improving Global Outcomes Acute Kidney Injury Work Group. *KDIGO Clinical Practice Guideline for Acute Kidney Injury*. Tech. rep. 1. Kidney Disease: Improving Global Outcomes, 2012, pp. 1–138. URL: https://kdigo.org/guidelines/acute-kidney-injury/.

[14] F. B. Taylor Jr. et al. “Towards Definition, Clinical and Laboratory Criteria, and a Scoring System for Disseminated Intravascular Coagulation.” In: *Thrombosis and Haemostasis* 86.5 (2001). Thieme publisher page exposed a PDF URL but returned HTML; The Blood Project public educational PDF matched title/authors/year and was used., pp. 1327–1330. DOI: 10.1055/s-0037-1616068. URL: https://doi.org/10.1055/s-0037-1616068.

[15] Sindhu Tipirneni and Chandan K. Reddy. “Self-Supervised Transformer for Sparse and Irregularly Sampled Multivariate Clinical Time-Series.” In: *ACM Transactions on Knowledge Discovery from Data* (2022). DOI: 10.1145/3516367. URL: https://doi.org/10.1145/3516367.

[16] Kyunghyun Cho et al. “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation.” In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*. 2014, pp. 1724–1734. DOI: 10.48550/arXiv.1406.1078. arXiv: 1406.1078. URL: https://arxiv.org/abs/1406.1078.

[17] Zhengping Che et al. “Recurrent Neural Networks for Multivariate Time Series with Missing Values.” In: *Scientific Reports* 8.6085 (2018). DOI: 10.1038/s41598-018-24271-9. URL: https://doi.org/10.1038/s41598-018-24271-9.

[18] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. “An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling.” In: *arXiv* (2018). DOI: 10.48550/arXiv.1803.01271. arXiv: 1803.01271 [cs.LG]. URL: https://arxiv.org/abs/1803.01271.

[19] Huan Song et al. “Attend and Diagnose: Clinical Time Series Analysis using Attention Models.” In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2018. URL: https://ojs.aaai.org/index.php/AAAI/article/view/11843.

[20] Satya Narayan Shukla and Benjamin M. Marlin. “Interpolation-Prediction Networks for Irregularly Sampled Time Series.” In: *International Conference on Learning Representations*. 2019. URL: https://openreview.net/forum?id=r1efr3C9Ym.

[21] Victor Chernozhukov et al. “Double/Debiased Machine Learning for Treatment and Structural Parameters.” In: *The Econometrics Journal* 21.1 (2018), pp. C1–C68. DOI: 10.1111/ectj.12097. URL: https://doi.org/10.1111/ectj.12097.

[22] Miruna Oprescu et al. “EconML: A Machine Learning Library for Estimating Heterogeneous Treatment Effects.” In: *NeurIPS 2019 Workshop on Causal Machine Learning*. 2019, pp. 1–6. URL: https://tripods.cis.cornell.edu/neurips19_causalml/.

[23] Stefan Wager and Susan Athey. “Estimation and Inference of Heterogeneous Treatment Effects using Random Forests.” In: *Journal of the American Statistical Association* 113.523 (2018), pp. 1228–1242. DOI: 10.1080/01621459.2017.1319839. URL: https://doi.org/10.1080/01621459.2017.1319839.

[24] Susan Athey, Julie Tibshirani, and Stefan Wager. “Generalized Random Forests.” In: *The Annals of Statistics* 47.2 (2019), pp. 1148–1178. DOI: 10.1214/18-AOS1709. URL: https://doi.org/10.1214/18-AOS1709.

[25] J. M. Smit et al. “Causal Inference Using Observational Intensive Care Unit Data: A Scoping Review and Recommendations for Future Practice.” In: *npj Digital Medicine* 6.1 (2023), p. 221. DOI: 10.1038/s41746-023-00961-1. URL: https://doi.org/10.1038/s41746-023-00961-1.

[26] Ioana Bica et al. “From Real-World Patient Data to Individualized Treatment Effects Using Machine Learning: Current and Future Methods to Address Underlying Challenges.” In: *Clinical Pharmacology & Therapeutics* 109.1 (2021), pp. 87–100. DOI: 10.1002/cpt.1907. URL: https://doi.org/10.1002/cpt.1907.

[27] Ilya Lipkovich et al. “Modern Approaches for Evaluating Treatment Effect Heterogeneity from Clinical Trials and Observational Data.” In: *Statistics in Medicine* 43.22 (2024), pp. 4388–4436. DOI: 10.1002/sim.10167. URL: https://doi.org/10.1002/sim.10167.

[28] Alicia Curth et al. “Using Machine Learning to Individualize Treatment Effect Estimation: Challenges and Opportunities.” In: *Clinical Pharmacology & Therapeutics* 115.4 (2024), pp. 710–719. DOI: 10.1002/cpt.3159. URL: https://doi.org/10.1002/cpt.3159.

[29] Theodore J. Iwashyna et al. “Implications of Heterogeneity of Treatment Effect for Reporting and Analysis of Randomized Trials in Critical Care.” In: *American Journal of Respiratory and Critical Care Medicine* 192.9 (2015), pp. 1045–1051. DOI: 10.1164/rccm.201411-2125CP. URL: https://doi.org/10.1164/rccm.201411-2125CP.

[30] Richard K. Crump et al. “Dealing with Limited Overlap in Estimation of Average Treatment Effects.” In: *Biometrika* 96.1 (2009). Duke university author PDF used., pp. 187–199. DOI: 10.1093/biomet/asn055. URL: https://doi.org/10.1093/biomet/asn055.

[31] Carlos Cinelli and Chad Hazlett. “Making Sense of Sensitivity: Extending Omitted Variable Bias.” In: *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 82.1 (2020). Author PDF used; first page states forthcoming JRSSB version and DOI., pp. 39–67. DOI: 10.1111/rssb.12348. URL: https://doi.org/10.1111/rssb.12348.

[32] Victor Chernozhukov et al. “Long Story Short: Omitted Variable Bias in Causal Machine Learning.” In: *Review of Economics and Statistics* (2026). Final publisher landing page exists, but publisher PDF returned 403/HTML; official arXiv author version was used. DOI: 10.1162/REST.a.1705. arXiv: 2112.13398. URL: https://doi.org/10.1162/REST.a.1705.

[33] Amit Sharma and Emre Kiciman. *DoWhy: An End-to-End Library for Causal Inference*. 2020. DOI: 10.48550/arXiv.2011.04216. arXiv: 2011.04216 [cs.AI]. URL: https://arxiv.org/abs/2011.04216.